



The Open University

M263 Building Blocks of Software

Unit 12

A case study

Study guide

This is largely a reading unit. You are not expected to carry out all the procedures discussed here. In particular, do not expect to grasp all the details of the implementation and tests discussed in Sections 3 and 4 on a first reading. We hope you find that you can attempt most of the activities here, and any assessment of this unit will be at a similar level. The details of the case study discussed here will not be required in the examination, and so they will not be given in the *Course Handbook*. The general skills used here — of putting a relation scheme into normal form and expressing informal conditions formally — are covered in earlier units in this block.

There is some practical computing associated with this unit. Running test code in the WorkPad is an integral part of the study of Section 4, and you should allow about 10% of your study time for the unit to do that. There are opportunities to do some practical work associated with Section 3, but this could be omitted if you are short of time.

This publication forms part of an Open University course. Details of this and other Open University courses can be obtained from the Course Information and Advice Centre, PO Box 724, The Open University, Milton Keynes, MK7 6ZS, United Kingdom: tel. +44 (0)1908 653231, e-mail general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by the Open University.

To purchase a selection of Open University course materials, please visit the Open University webshop at www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; e-mail ouwenq@open.ac.uk

The Open University, Walton Hall, Milton Keynes, MK7 6AA.

First published 2004. Second edition 2008.

Copyright © 2008 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd of 90 Tottenham Court Road, London W1T 4LP.

Edited, designed and typeset by The Open University, using the Open University T_EX System.

Printed and bound in the United Kingdom by The Charlesworth Group, Wakefield.

ISBN 978 07492 2304 5

Contents

Introduction	4
1 Setting the scene	5
2 The underlying database	9
2.1 The relation schemes	10
2.2 The corresponding classes	13
2.3 Adding an appointment	15
3 The classes Practice and PracTrans	20
4 Checking against the specification	26
Solutions to the exercises	38
Appendix: Class specifications	42
Index	48

Introduction

The purpose of this unit is to bring together ideas that have been introduced in this block, indeed in the course so far, and show how they can be used in the process of developing and testing a small software system. We do this by considering a case study about the development of an appointment system for a medical practice.

The unit will take the case study through from an initial discussion of requirements to an implementation in the WorkPad that can be tested against the requirements. Of course, a real implementation of such a system would use more sophisticated software with many more features than we have available. However, the process of formalising requirements and using them to devise tests for any implementation would still be applicable.

A case study of this sort means that some of the classes we implement will be quite complex and have a considerable number of methods. You will not be expected to remember these methods and will only need to refer to them in the context of this unit.

Developing a piece of software of any complexity requires constant interchange between the client and the development team. During this interchange it is likely that the client's requirements will change. Documenting and formalising requirements is part of this interchange and should help it go smoothly. Although the client cannot be expected to read the requirements when written using formal logic and predicates, the natural language versions should help ensure that the client knows what is being catered for.

The final section of the unit will be about testing the implementation. By then we will have a variety of **transactions** (we use this term for interactions between a user and the system that we wish the software to enable), each of which will have preconditions and postconditions. We will look at how we can devise code that can test these transactions. Although this testing will not prove that the implementation is correct, it should be designed to help us to check that all essentially different cases have been covered.

We hope that after you have read this unit you will appreciate the value of establishing requirements reasonably formally and of writing down preconditions and postconditions of transactions. Working this way, rather than rushing into code, should result in an application that is what the client requires and that is reasonably free from errors.

In Section 1, we describe what the software is intended to achieve and start identifying and formalising the requirements. The software is essentially working with a database, and in Section 2 we discuss the structure of this database and identify appropriate relation schemes and the associations between them. At this point we would normally implement the software using a standard database package that enabled us to develop the required user-defined applications. It is inappropriate to introduce such a package at this stage in the course, and so we shall implement the case study in the WorkPad. This implementation is described in Section 3. Although we will not be able to implement all the features of a database, we will be able to demonstrate the required user-defined applications. In the final section we will show how the formal descriptions of the requirements makes it possible to test the user-defined applications reasonably systematically.

1 Setting the scene

In this unit we discuss the processes necessary for developing software for an appointment system at a medical centre. This is a more complex situation than anything considered earlier in the course. We will start by giving some information about the appointment system in informal language. We will then investigate how these requirements can be formalised using predicates.

The medical centre contains medical workers, who we will refer to as *members* of the practice. At this time the members consist of doctors, nurses and physiotherapists. Patients can make appointments with a doctor, a physiotherapist or a nurse. The following points should be noted.

A patient cannot make an appointment with the physiotherapist without a note from the doctor.

A patient is offered the first available appointment but can refuse it.

Not all members are available for all appointments.

Appointments can be cancelled.

Appointments to see a nurse or a doctor are allocated the same length of time.

Appointments to see a physiotherapist are allocated double the length of time of an appointment with a doctor or nurse.

Members of the practice may also be patients of the practice.

The aim of this unit is to design software that will enable appointments to be offered and made. The appointments that are offered should not clash with any appointments already made and should take into account availability of individual members.

The next step is to introduce some predicates that will help us formalise the requirements. In order to do this we need some predefined sets. These include:

the set *Patient*, which contains the patients of the practice;

the set *Member*, which contains the members of the practice, i.e. the doctors, physiotherapists and nurses.

We need to distinguish between members and patients as we need to keep a record of members' availability. Another reason to distinguish between the two sets is that there may well be other transactions that differ between them. Hence the two sets are likely to correspond to two data types. Other sets that we shall need include:

Appointment, which contains the current appointments;

Action, which gives the possible activities of members.

A member may have an appointment as a member, or as a patient, or may be unavailable, or may be free. The set *Action* will give these possible activities.

We need to decide how we are going to represent the times at which appointments can be made. Appointments are made for a fixed length of time (for doctors and nurses), or double this time (for physiotherapists). So we can represent possible appointment times using numbered time slots within each day. To represent possible appointment slots we introduce two sets, *Day* and *TimeSlot*. At this stage of the process we avoid indicating how we

will implement these sets. It is good practice to defer detailed implementation decisions until a later stage in the development process.

The following list of predicates should help us to start expressing the requirements more formally.

ISDOCTOR(*aMem*): *aMem* is a doctor of the practice.

ISNURSE(*aMem*): *aMem* is a nurse of the practice.

ISPHYSIO(*aMem*): *aMem* is a physiotherapist of the practice.

APPOINTMENT(*aMem*, *aPat*, *d*, *t*): the member *aMem* has an appointment on day *d* in time slot *t* with the patient *aPat*.

ACT(*aMem*, *d*, *t*, *act*): the member *aMem* has activity *act* on day *d* in time slot *t*.

PHYSIONOTE(*aPat*): the patient *aPat* has a note to see the physiotherapist.

A feature of this particular medical practice is that members of the practice may themselves be patients of the practice. This is an important issue when making appointments. Information about this is accessed using the predicate below.

ISPATIENT(*aMem*): the member *aMem* is a patient of the practice.

It is useful to express the requirements more formally, but still in natural language at this stage. We will start with a list of propositions about the practice.

1. The medical practice has three types of member: doctor, nurse and physiotherapist.
2. Appointments are for one member of the practice to see one patient at a particular time.
3. Patients may or may not have a note for the physiotherapist.
4. Patients cannot see the physiotherapist unless they have a note.
5. Members of the practice may be patients but can never have an appointment with themselves.
6. A member may be unavailable for a particular appointment slot.
7. A record is held as to whether an appointment has been kept.
8. A record is held of medical notes for each appointment.
9. Appointments are for a fixed length of time. The length of time for an appointment with a physiotherapist is twice the length of time for an appointment with either a nurse or doctor.

Any member of the practice must be one (and only one) of these three types.

We now look at how propositions about the practice may be expressed formally. We shall do this using the predicates given above and the following predefined sets: *Patient*, *Member*, *Appointment*, *Action*, *Day* and *TimeSlot*. As a first example, consider the following proposition.

Patients cannot see the physiotherapist unless they have a note.

To express this proposition formally, we might see it as follows.

If a patient has an appointment with a physiotherapist then they have a note for a physiotherapist.

We can express “a patient (*aPat*) has an appointment with a physiotherapist (*aMem*)” formally as follows.

$\exists d \text{ in } Day [\exists t \text{ in } TimeSlot [APPOINTMENT(aMem, aPat, d, t) \wedge ISPHYSIO(aMem)]]$

If this is true, then the patient must have a note for the physiotherapist. We can express “if ... then ...” using the connective $\Rightarrow$. The resulting predicate must be true whoever the patient *aPat* and member *aMem* may

be, which can be expressed using the quantifier $\forall$. We get the following formal proposition.

$$\forall aPat \text{ in } Patient [\forall aMem \text{ in } Member \\ [(\exists d \text{ in } Day [\exists t \text{ in } TimeSlot \\ [APPOINTMENT(aMem, aPat, d, t) \wedge ISPHYSIO(aMem)])] \Rightarrow PHYSIONOTE(aPat)]]]$$

There are other, equivalent, propositions that we could use to express formally the given condition about the practice.

This comment is true of most formal propositions. We will not give all the equivalent forms of formal propositions.

Activity 1.1

Express formally the proposition below.

The medical practice has three types of member: doctor, nurse and physiotherapist.

Discussion We can say that, for every member, that member is either a doctor, a nurse or a physiotherapist.

$$\forall aMem \text{ in } Member [ISNURSE(aMem) \vee ISDOCTOR(aMem) \vee ISPHYSIO(aMem)]$$

We look next at requirement 2 in the list above.

Appointments are for one member of the practice to see one patient at a particular time.

This is quite a complicated requirement to express formally. It is also extremely important, as it is designed to avoid conflicting appointments.

We will give two formal propositions. The first will assert that there does not exist an appointment time when a member has appointments with two different patients.

$$\neg(\exists d \text{ in } Day [\exists t \text{ in } TimeSlot [\exists aMem \text{ in } Member \\ [\exists aPat1 \text{ in } Patient [\exists aPat2 \text{ in } Patient [APPOINTMENT(aMem, aPat1, d, t) \\ \wedge APPOINTMENT(aMem, aPat2, d, t) \wedge (\neg(aPat1 == aPat2))]]]]])]$$

The second asserts that no patient has two appointments at the same time.

$$\neg(\exists d \text{ in } Day [\exists t \text{ in } TimeSlot [\exists aPat \text{ in } Patient \\ [\exists aMem1 \text{ in } Member [\exists aMem2 \text{ in } Member [APPOINTMENT(aMem1, aPat, d, t) \\ \wedge APPOINTMENT(aMem2, aPat, d, t) \wedge (\neg(aMem1 == aMem2))]]]]])]$$

For requirement 2 to be satisfied we need both these conditions to hold.

Members of the practice may also be patients, as we noted earlier. This could cause another sort of clash — namely, a patient being allocated an appointment with themselves as doctor, say. We need to rule this out. To be able to do this, we need a way of obtaining the patient details of a member. To do this, we shall assume that we have a function *PAT*, with input set *Member* and output set *Patient*. Given a member of the practice *aMem* who is also a patient, *PAT(aMem)* is the patient associated with the given member.

If a member is also a patient of the practice, then two items will be recorded for the member — one in the set *Patient* and one in the set *Member*. These items represent the different roles: as member and as patient.

Activity 1.2

Using the predefined sets and predicates given earlier, and the function *PAT*, express the following requirement formally.

Members of the practice may be patients but can never have an appointment with themselves.

Discussion We need to say that there does not exist a time when a member has an appointment with him- or herself.

$$\neg(\exists d \text{ in } Day[\exists t \text{ in } TimeSlot[\exists aMem \text{ in } Member[APPOINTMENT(aMem, PAT(aMem), d, t)]]])$$

Activity 1.3

Spend a few minutes thinking about whether there are any circumstances not covered so far that could cause a clash of appointments.

Discussion One circumstance that we have not covered is that, since members of the practice may be patients, it is possible for a member to have an appointment as a patient at a time when they have an appointment as a member. You may well have thought of others.

Expressing the requirements formally is quite challenging. However, the thinking that goes into this task is essential if the software is going to work as required. It is much easier to write processes that cover all the requirements once those requirements have been made precise. Such formal statements will also make it easier to test that the requirements have been satisfied.

In order to complete the formal requirements we need to specify what transactions should be available. Before reading on, spend a few moments thinking about this, then look at our list below.

1. Members and patients can be added to the practice.
2. Patients can be recorded as having or as not having a note for the physiotherapist.
3. Members can be made unavailable for particular time slots.
4. The next available appointment, at or after a given time slot and with a particular type of member, can be found. The appointment found will not break any of the requirements.
5. An appointment can be added to the practice.
6. A list of appointments for a member can be returned.
7. A list of appointments for a patient can be returned.
8. Medical notes about an appointment can be added.
9. A record of whether an appointment has been kept can be added.
10. An appointment can be cancelled.
11. Details of an appointment for a given time and patient can be found.
12. Details of an appointment for a given time and member can be found.
13. A list of all the appointments can be returned.
14. A list showing the actions of the members at different times can be returned.

You may have a different list. You may well have more transactions. In practice, the transactions that are required will emerge from discussions with the client.

At this stage the transaction descriptions are expressed in natural language and are informal. We will see how to formalise some of them when we have established the classes involved in our software.

Objectives for Section 1

After studying this section you should be able to do the following.

- ☐ Appreciate the need for identifying requirements precisely.
- ☐ Express simple requirements formally using given predicates and pre-defined sets.

Exercises on Section 1

Exercise 1.1

Express formally each of the following propositions.

- (a) If a patient $aPat$ has an appointment with a member $aMem$ on day d at time t , and $aMem$ is a physiotherapist and t is an odd number, then $aPat$ has an appointment with $aMem$ on day d at time $t + 1$. (*Hint*: recall that an integer is odd if its remainder when divided by 2 is 1.)
- (b) There is a nurse and a day such that the nurse has no appointments on the given day.

Assume that t is of type **Int**. You will see later how this is connected to the way we will make appointments with a physiotherapist.

2 The underlying database

Our appointment software will be designed in three layers, each with corresponding classes. The top level will enable a user to perform the transactions discussed in Section 1. The bottom level stores the data we need about members, patients and appointments. We can see this bottom level as an underlying database, and can use the ideas developed in Unit 11 to discuss the structure of this database. We shall then describe classes corresponding to each of the schemes in this database. The intermediate level of the overall design enables the bottom level classes to communicate with each other in a way that respects various rules they must obey (such as keys and functional dependencies). It needs to have sufficient methods to enable a user to perform the required top level transactions.

The first step is to decide on some relation schemes that we will need and to identify the functional dependencies in those schemes and the relationships between them. To carry out this step formally is beyond the scope of this course, so we will deal with it fairly briefly and intuitively.

A common temptation at this stage is to try to be too elaborate and to introduce features that were not in the original requirements. In this unit, we will only discuss the practice database as it affects appointments.

Before reading on, spend a few moments deciding on some possible relation schemes for our database.

2.1 The relation schemes

It seems sensible to have separate relation schemes concerning members of the practice, patients of the practice, and appointments. We look first at a scheme for patients, **PATIENT**.

PATIENT			Relation scheme
Attribute	Domain	Type	
<i>patientID</i>	<i>PatientIDCode</i>	mString	
<i>patientInfo</i>	<i>SeqOfChar</i>	mString	
<i>physioNote</i>	<i>Bool</i>	Bool	
<i>Functional dependencies</i>			
FD1: { <i>patientID</i> } $\rightsquigarrow$ { <i>physioNote</i> , <i>patientInfo</i> }			

This scheme has an attribute, *patientID*, giving a unique identity code to each patient. For the purposes of this case study we just have one attribute, *patientInfo*, to record details such as name and address. In practice, this would normally be replaced by several attributes. The attribute *physioNote* records whether the patient has a note to see a physiotherapist.

The scheme for members, **MEMBER**, has an attribute, *memberID*, giving a unique identity code to each member, and a single attribute, *memberInfo*, to record details similar to those for patients. Members also have an attribute recording whether they are a doctor, nurse or physiotherapist. This is stored in *memberType*, using an appropriate character. We also need to link those members who are patients to the scheme **PATIENT**. We will do this by giving members an attribute that records their *patientID* if they are a patient in the practice. If a member is not a patient of the practice then there will be no *patientID* to store. In such a case, a value **null** will be stored. This possibility is reflected in the domain given below for the attribute *PatientID*.

MEMBER			Relation scheme
Attribute	Domain	Type	
<i>memberID</i>	<i>MemberIDCode</i>	mString	
<i>memberType</i>	{‘D’, ‘N’, ‘P’}	Char	
<i>memberInfo</i>	<i>SeqOfChar</i>	mString	
<i>patientID</i>	<i>PatientIDCode</i> $\amalg$ { null }	mString	
<i>Functional dependencies</i>			
FD2: { <i>memberID</i> } $\rightsquigarrow$ { <i>memberType</i> , <i>memberInfo</i> , <i>patientID</i> }			

The notation $\amalg$ was introduced in Unit 1, and represents the disjoint union of two sets. You can read it as “or”. The type **mString** does not contain a value **null**, so we will need to choose a particular value within this type to represent the value **null**. (It is possible to describe a class construction to reflect the idea of a disjoint union, but to do so is beyond the scope of this course.)

The relation scheme for appointments, **APPOINTMENT**, requires attributes giving the day and time slot, and also the ID codes of the member and patient involved in the appointment. We now need to decide how we will represent the day and time slot. For simplicity we will use integers. We also include two attributes that were required by the specification, namely an attribute, *kept*, indicating whether the appointment was kept, and an attribute, *note*, recording any medical notes from the appointment.

APPOINTMENT			Relation scheme
Attribute	Domain	Type	
<i>day</i>	<i>Day</i>	Int	
<i>time</i>	<i>TimeSlot</i>	Int	
<i>memberID</i>	<i>MemberIDCode</i>	mString	
<i>patientID</i>	<i>PatientIDCode</i>	mString	
<i>kept</i>	<i>Bool</i>	Bool	
<i>notes</i>	<i>SeqOfChar</i>	mString	
<i>Functional dependencies</i>			
FD3: $\{day, time, memberID\} \rightsquigarrow \{patientID, kept, notes\}$			
FD4: $\{day, time, patientID\} \rightsquigarrow \{memberID, kept, notes\}$			

Take a moment to look back at our initial list of requirements, given on page 6. Can you see anything that we have not considered when describing the relation schemes above?

There is one thing missing. At the moment there is no way of knowing whether a member is free at a proposed time for an appointment. One of our transactions was that “Members can be made unavailable for particular time slots”. We can check whether a member already has an appointment (either as member or as a patient), but, with the schemes described so far, we cannot check whether a member is unavailable for some other reason.

This observation shows the need for some form of diary. If this was a paper diary then members would enter in it both their appointments and any other times when they are unavailable. If their plans are altered or appointments cancelled then the corresponding entry would be crossed out. We will use this idea. We introduce a relation scheme called **DIARYENTRY** with attributes *day*, *time*, *memberID* and *act*, where *act* is a character indicating the member’s actions at that time.

DIARYENTRY			Relation scheme
Attribute	Domain	Type	
<i>day</i>	<i>Day</i>	Int	
<i>time</i>	<i>TimeSlot</i>	Int	
<i>memberID</i>	<i>MemberIDCode</i>	mString	
<i>act</i>	<i>Action</i>	Char	
<i>Functional dependencies</i>			
FD5: $\{day, time, memberID\} \rightsquigarrow \{act\}$			

It will be useful to specify the elements of *Action* at this stage. A reasonable choice is $\{‘F’, ‘A’, ‘P’, ‘U’\}$ where:

‘F’ means “is free”;

‘A’ means “has an appointment as a member of the practice”;

‘P’ means “has an appointment as a patient of the practice”; and

‘U’ means “is unavailable”.

A relation over this scheme acts as a diary. When a new entry is made the *act* attribute is set to ‘A’, ‘P’ or ‘U’. If a member subsequently becomes

available then *act* is set to ‘F’ for “free”. This latter action corresponds to crossing out a diary entry. Thus any relation on **DIARYENTRY** contains only entries and crossed out entries. We can infer from the absence of an entry for a particular day and time slot that a member is available then.

We now have four relation schemes. It is sensible to check that they are all in third normal form and to clarify the links between them. We first recall the definition of third normal form from Unit 11.

A relation is in **third normal form** if it is in second normal form and has no transitive dependencies.

A relation scheme is in **second normal form** if, for any key K and any attribute x not in K , there is no proper subset P of K that functionally determines x .

We will not start by filling the diary with entries saying members are free. Members are explicitly recorded as free only where there was a previous entry indicating they were otherwise engaged. Keeping the “crossed out” entries provides a useful audit trail.

In particular, any relation scheme with only single attribute keys will be in second normal form.

Activity 2.1

For each of the schemes **MEMBER**, **PATIENT**, **APPOINTMENT** and **DIARYENTRY**:

- (a) give all its keys;
- (b) explain why the scheme is in second normal form;
- (c) explain why the scheme is in third normal form.

Discussion We look first at **MEMBER**, which has $\{memberID\}$ as its only key. This can be seen from the functional dependencies of the scheme. The set of attributes $\{memberID\}$ has the determining property, and is bound to be minimal since it contains only one member. Since the key contains just one attribute, the scheme is bound to be in second normal form. There are no transitive dependencies (since the scheme has only one functional dependency), and so the scheme is in third normal form.

The scheme **PATIENT** has $\{patientID\}$ as its only key, and is in third normal form for reasons similar to those given for **MEMBER**.

The scheme **APPOINTMENT** has two keys: $\{day, time, memberID\}$ and $\{day, time, patientID\}$. No attribute is dependent on a proper subset of either key, so it is in second normal form. There are no transitive dependencies. Hence this scheme is in third normal form.

Finally, the scheme **DIARYENTRY** has one key, $\{day, time, memberID\}$. No attribute is dependent on a subset of this key, so it is in second normal form. It has no transitive dependencies and so is also in third normal form.

$\{patientID\}$ cannot be a key for **MEMBER** as it is sometimes **null** and so does not determine the other attributes.

For **APPOINTMENT**, we need to declare a primary key. This will be $\{day, time, memberID\}$.

There is a one to many relationship between **PATIENT** and **APPOINTMENT**. The foreign key is $\{patientID\}$ in **APPOINTMENT**. There is a one to many relationship between **MEMBER** and **APPOINTMENT**. The foreign key is $\{memberID\}$ in **APPOINTMENT**. There is a one to many relationship between **MEMBER** and **DIARYENTRY**. The foreign key is $\{memberID\}$ in **DIARYENTRY**.

There is also a relationship between **MEMBER** and **PATIENT** but this is not a standard one as described in Unit 11. In database software *patientID* would be declared as a foreign key in **MEMBER** referencing **PATIENT**.

Where *patientID* is not **null**, we want to link the member to the corresponding patient. If *patientID* is **null** then there is no corresponding patient, which complicates matters.

The next step is to implement the database. This means storing the details of the relation schemes, the way they are linked via foreign keys, and any extra constraints that are not shown by the primary keys. Once this has been implemented we need to add the data and find ways of carrying out the required transactions on the data. There are standard transactions that are used, for example adding and deleting records, updating records and extracting information from the stored records. These transactions are sufficiently common for a special database language, known as SQL (Structured Query Language) to have been developed that provides the tools enabling us to carry them out. This language is usually part of the implementation of specialist database software. However, working with such software is beyond the scope of this course.

To see how the formal specification of requirements can help with implementing and testing software, we shall implement a small prototype of our appointment software in the WorkPad. Although this implementation is necessarily constrained by what is available to us in the WorkPad and, as a result, may differ in many respects from commercially available software, it will illustrate how many of the techniques introduced so far in the course can help us develop and test quite a complex piece of code that satisfies some given requirements. This implementation is in several stages. The first is to implement classes in the WorkPad corresponding to each of our relation schemes, and we look at this next.

2.2 The corresponding classes

The implementation introduces a class for each relation scheme. The data set of each of these classes is conveniently described in terms of its attributes, and we shall describe the data set in terms of the way it is implemented. Look, for example, at the relation scheme **PATIENT**. That scheme has three attributes, and the type of each is specified. Each of these types is available in the WorkPad. A natural implementation of the data set of the corresponding class is to use three corresponding attributes.

The only methods that we shall require in the corresponding class are methods to get and to set each attribute. We shall name these in a natural way: for example, *getPatientID* will return the value of the attribute *patientID*, while *setPhysioNote(aVal)* will change the state of the receiving object by changing the value of the attribute *physioNote* to *aVal*, and leaving the values of the other attributes unchanged.

The class **Patient** is specified below. The default value may be regarded as acting as a **null** value for the class — no actual patient will be given the empty string for their *patientID*.

Patient	Class specification
<i>Attributes</i> <i>patientID</i> in mString; <i>patientInfo</i> in mString; <i>physioNote</i> in Bool <i>Default</i> <i>patientID</i> = ""; <i>patientInfo</i> = ""; <i>physioNote</i> = false Methods to get and to set each attribute.	

The classes corresponding to the other schemes will be specified in a similar way. There are a few points to note. The domain of the attribute *patientID* in **MEMBER** includes the possibility of a value **null**. As indicated above, we shall not give any actual patient the empty string for their *patientID*, and so can take the empty string to represent the **null** value of this attribute. Again, the default value for the class **Member** can be seen as a **null** value for the class. No actual member will be given an empty string as their *memberID*, nor the value 'X' for the attribute *memberType* (this represents a situation where no type — nurse, doctor or physiotherapist — has yet been given for a member).

Member	Class specification
<i>Attributes</i> <i>memberID</i> in mString; <i>memberType</i> in Char; <i>memberInfo</i> in mString; <i>patientID</i> in mString <i>Default</i> <i>memberID</i> = ""; <i>memberType</i> = 'X'; <i>memberInfo</i> = ""; <i>patientID</i> = "" Methods to get and to set each attribute.	

Each of the classes **Appointment** and **DiaryEntry** has an initialisation method, which for convenience allows attributes to be set at the same time. Again, the default value for each class can be seen as a **null** value.

We have not implemented the relationships between the classes at this stage.

Appointment	Class specification
<i>Attributes</i> <i>day</i> in Int; <i>time</i> in Int; <i>memberID</i> in mString; <i>patientID</i> in mString; <i>kept</i> in Bool; <i>notes</i> in mString <i>Default</i> <i>day</i> = 0; <i>time</i> = 0; <i>memberID</i> = ""; <i>patientID</i> = ""; <i>kept</i> = false; <i>notes</i> = "" <i>Methods</i> <i>init</i>(<i>d</i> in Int, <i>t</i> in Int, <i>memID</i> in mString, <i>patID</i> in mString) state change Methods to get each attribute. Methods to set each of the attributes <i>patientID</i>, <i>kept</i> and <i>notes</i>.	

Recall that for simplicity we avoided introducing a **Date** class and implemented *day* and *time* as integers.

DiaryEntry	Class specification
<i>Attributes</i> <i>day</i> in Int; <i>time</i> in Int; <i>memberID</i> in mString; <i>act</i> in Char <i>Default</i> <i>day</i> = 0; <i>time</i> = 0; <i>memberID</i> = ""; <i>act</i> = 'F' <i>Methods</i> <i>init</i>(<i>d</i> in Int, <i>t</i> in Int, <i>memID</i> in mString) state change Methods to get each attribute, and a method to set <i>act</i>.	

2.3 Adding an appointment

The next level in our design will enable the classes described in the previous subsection to communicate with each other. We shall discuss a class to do this, called **Practice**, in the next section. This class also needs to have sufficient methods to enable the transactions discussed in Section 1 to be performed. Our design for the appointment software will involve a single instance of the class **Practice**. The state of this object will include all the information stored in our underlying database — which will include an implementation of a relation over each of our four schemes.

Many of the methods of the class **Practice** will be relatively straightforward. We will need methods to add a tuple to each of the underlying relations, and to delete a tuple from each of these relations; to set attributes within tuples, and to examine data — either entire relations, or particular tuples or attributes. For example, there is a method *setPhysioNote* with parameters *aPat* in **Patient** and *aNote* in **Bool** that permits one to set the value of the attribute *aNote* for the patient *aPat*, and so to record whether the patient has a note for the physiotherapist. This method is a state change method of the **Practice** class. It has precondition **true**. We can express the postcondition as $PHYSIONOTE(aPat) == aNote$.

The predicate *PHYSIONOTE* was introduced in Section 1.

The methods of **Practice** need to take account of the structure of the underlying relational database — functional dependencies of the schemes and the relationships between them. We also need to be aware of any features of the original problem that are not reflected within any of the schemes. Such considerations can lead to some of the methods being far from straightforward to specify. We shall concentrate here on a vital method of **Practice**, *addAppt*, which adds a new appointment. This is complicated, and we shall break it down into several parts.

A new appointment will involve a member, a patient, a time and a day. Before we add an appointment, we need to be sure that it will satisfy our requirements. For example, we would not want to add an appointment when the patient is already booked for another appointment. Before reading on, spend a few moments thinking about conditions that must hold before we add a new appointment. Such conditions are listed below, and will be brought together in the preconditions for the *addAppt* method.

We will make some decisions at this stage. The first is that the system will check whether a patient has another appointment at the proposed time, but will not have the information to check that the patient is not already engaged in some other way. The second decision is that we will implement the requirement that appointments with a physiotherapist are double the length of normal appointments by making two consecutive appointments when the member involved is a physiotherapist. In addition, to ensure that we do not leave gaps in a physiotherapist's schedule, all appointments with a physiotherapist will start on an odd-numbered time slot.

When a corresponding transaction is executed, the person executing it will need to check with the patient whether they are available before adding the appointment.

Preconditions for the method addAppt.

1. The patient does not have another appointment on the same day and in the same time slot as the appointment.
2. The member does not have another appointment on the same day and in the same time slot as the appointment.
3. The member does not have another appointment, as patient, on the same day and in the same time slot as the appointment.
4. The member is not otherwise occupied on the same day and in the same time slot as the appointment.

5. The patient involved is not the member involved.
6. The day and time for the appointment are not too large.
7. The appointment is not **null**.

Now appointments with a physiotherapist require two time slots, so we need to check that the time slot is not the last one of the day and that the first four conditions hold for the succeeding time slot. If the member is a physiotherapist, we need conditions 8-14 to hold.

8. The patient has a note for the physiotherapist.
9. The time slot has an odd number.
10. The time slot is not the last one in the day.
11. The patient does not have another appointment on the same day and in the time slot after the appointment.
12. The member does not have another appointment on the same day and in the time slot after the appointment.
13. The member does not have another appointment, as patient, on the same day and in the time slot after the appointment.
14. The member is not otherwise occupied on the same day and in the time slot after the appointment.

If the diary (the relation based on **DIARYENTRY**) is updated correctly, then checking it should cover both points 4 and 2 above. However, this is something to think about when implementing rather than when describing the precondition.

We now look at how we can specify the method *addAppt* formally. This is a complicated task. The list above breaks down the precondition into manageable parts. To express the precondition formally requires that each of the above conditions is expressed formally and then all of them are connected by $\wedge$. We will consider formal versions of just a few of the conditions above.

To express details of specifications of methods of **Practice** formally we can use two things:

1. the classes **Member**, etc, introduced in the previous subsection;
2. the predicates introduced in Section 1 (which we take as “primitives” — used to mediate between the formalism and the outside world).

To express formally the first condition above, we break the problem into stages by first introducing a new predicate.

$PATFREE(d, t, aPat): \neg \exists aMem \text{ in } Member[APPOINTMENT(aMem, aPat, d, t)]$

In the predicate *APPOINTMENT*, we will take *aMem* to be an object of type **Member**, and *aPat* to be an object of type **Patient**. Now the class **Appointment** uses member and patient ID codes rather than actual members or patients. This is not a problem since we can find the member and patient from these ID codes (which in each case form a key of the corresponding relation scheme). For now, we shall assume that we have available a function *findPatient* whose input is a patient’s ID code, and which returns the corresponding patient object, and similarly a function *findMember* whose input is a member’s ID code, and which returns the corresponding member object.

The method *addAppt* will have a single parameter, of type **Appointment**, which we shall denote as *appt*. To express details of the appointment, we can use methods of the class **Appointment** such as *getDay* and *getTime*, which return the day and time slot of an appointment.

See item 2 on page 20.

We would not want to add a **null** appointment.

Preconditions and postconditions have two roles: the first as a message to the implementer as to what they can assume and what they must ensure; the second to any tester as to what to check for. Thus preconditions and postconditions should be implementation-independent as far as possible.

We will assume that these are now available.

The class **Practice** has methods with a similar effect, but those need to be addressed to an object of type **Practice**. The functions we use here relate to “the practice under discussion”, which has not yet been turned into an object.

The first condition listed above was this.

The patient does not have another appointment on the same day and in the same time slot as the appointment.

We can now express this formally as follows.

$PATFREE(appt.getDay(), appt.getTime(), findPatient(appt.getPatientID()))$

Next, we consider the second condition above.

The member does not have another appointment on the same day and in the same time slot as the appointment.

To express this formally, we observe that it asserts that there should not already be an appointment, different from *appt*, on the same day, in the same time slot, and involving the same member.

Activity 2.2

Suppose that *appt* is the appointment that is to be added, and that *appt1* is another appointment. Express the following statements formally. (Use the methods *getDay*, *getTime* and *getMemberID* of **Appointment**, and the method *findMember(memID)* of **Practice**, that returns the member with ID code *memID*.)

You can assume here that all the types concerned have an equality operation expressed as `==`.

- (a) (i) The members involved in *appt1* and *appt* are the same.
- (ii) The days involved in *appt1* and *appt* are the same.
- (iii) The time slots involved in *appt1* and *appt* are the same.
- (b) Express formally the condition below.

The member does not have another appointment on the same day and in the same time slot as the appointment *appt*.

Discussion

- (a) (i) $findMember(appt.getMemberID()) == findMember(appt1.getMemberID())$.
- (ii) $appt.getDay() == appt1.getDay()$.
- (iii) $appt.getTime() == appt1.getTime()$.
- (b) We need to assert that there is no appointment *appt1* satisfying the three conditions in part (a), combined using $\wedge$, and different from *appt*.

$$\neg(\exists appt1 \text{ in } Appointment \\ [(findMember(appt.getMemberID()) == findMember(appt1.getMemberID())) \\ \wedge (appt.getDay() == appt1.getDay()) \\ \wedge (appt.getTime() == appt1.getTime()) \wedge \neg(appt == appt1)])$$

The other preconditions listed above can be formalised in a similar way. We shall not look at the details. We shall express the precondition of *addAppt* using another new predicate, *APPOINTMENTOK(appt)*. This will assert that *appt* satisfies all the preconditions above, and can be formalised by expressing each condition formally, then combining them all using $\wedge$.

This means we have a succinct expression for the precondition of *addAppt*. What about the postcondition? To express formally the proposition

The appointment list contains the appointment *appt*

we can use the predicate *APPOINTMENT* from page 6, as below.

$APPOINTMENT(findMember(appt.getMemberID()), findPatient(appt.getPatientID()), appt.getDay(), appt.getTime())$

Activity 2.3

Are there any additional conditions that you would want to include to the postcondition of the method *addAppt*?

Discussion We would wish to ensure that a diary entry for the member had been made at the appointment time. Since we are implementing the requirement that physiotherapists' appointments are twice the length of normal appointments by making double appointments, if the member is a physiotherapist then we need to add an appointment at the time after the one input, and to add an appropriate diary entry for the member.

In *addAppt*, we assume that the patient in the appointment is not a member. We shall deal with that case in the next section.

We can now give a formal description of the method *addAppt*. This includes formal expressions of the two additional conditions identified in the preceding activity. The predicate *ACT* is used to assert that diary entries for members must be added or updated. The final part of the postcondition ensures that an appointment with a physiotherapist is given two time slots. In the formal postcondition below: *d* stands for *appt.getDay()*; *t* stands for *appt.getTime()*; *theMem* stands for *findMember(appt.getMemberID())* and *thePat* stands for *findPatient(appt.getPatientID())*.

method *addAppt*(*appt* in **Appointment**) **state change**

pre *APPOINTMENTOK*(*appt*).

post *APPOINTMENT*(*theMem*, *thePat*, *d*, *t*)

$\wedge ACT(theMem, d, t, 'A') \wedge$

$(ISPHYSIO(theMem) \Rightarrow$

$(APPOINTMENT(theMem, thePat, d, t + 1) \wedge ACT(theMem, d, t + 1, 'A'))$).

Another method of **Practice** that is crucial to making appointments is one that finds a suitable appointment. We will specify a method *findAppt* that finds the first available appointment with a member of the required type that occurs at or after a given day and time slot. We discuss this method in the next two activities.

Activity 2.4

Suggest parameters that the method *findAppt* should have, and preconditions that the parameters should satisfy.

Discussion Necessary parameters are the day and time slot and the member type required, as well as the patient who requires an appointment.

We will assume that the patient is identified using their ID *patID*, and the type of the other parameters is that of the corresponding attribute in the classes **Appointment** and **Member**: *day* in **Int**; *time* in **Int** and *memberType* in **Char**. We then require that:

there is a patient of the practice with identifier *patID*, and
day > 0, *time* > 0, and *memberType* ∈ {'D', 'N', 'P'}.

To express the postcondition of *findAppt* formally, it is useful to introduce another predicate. This says that the time slot (*d1*, *t1*) occurs at or after the time slot (*d*, *t*).

$$AFTER(d1, t1, d, t) : ((d1 == d) \wedge (t1 \geq t)) \vee (d1 > d)$$

Activity 2.5

Suppose that the method call *findAppt(d, t, type, patID)* returns a value *appt* of type **Appointment**. Express formally three conditions that *appt* must satisfy. As well as suitable methods, you can use the predicates *AFTER* and *APPOINTMENTOK*.

Discussion The three conditions that must hold are that:
appt can be added to the appointment list without breaking any rules, which is summarised in the proposition *APPOINTMENTOK(appt)*;
appt occurs at or after the time specified in the parameters *d* and *t*, i.e. *AFTER(appt.getDay(), appt.getTime(), d, t)*;
appt involves a member of the type specified in the parameter *type*, i.e. *findMember(appt.getMemberID()).getMemberType() == type*.

The postcondition of *findAppt* is that an instance *appt* in **Appointment** is returned and that either all three conditions in the discussion of Activity 2.5(a) are satisfied, or *appt* is **null** and there is no appointment in *Appointment* satisfying these three conditions. (It would be tempting just to say that: either *appt* satisfies these conditions, or it is **null**. This would be a mistake, however, since a method that always returned **null** appointments would then satisfy the postcondition.)

Objectives for Section 2

After studying this section you should be able to do the following.

- ☐ Give the parameters, precondition and postcondition of a method of **Practice** described informally.
- ☐ Express formally conditions relating to the type **Practice**.

Exercises on Section 2

Exercise 2.1

Suggest the parameters for a method *setKept* of the class **Practice** that enables one to record whether a given appointment has been kept.

Exercise 2.2

Express formally the condition below (*appt* is of type **Appointment**).

If the member is a physiotherapist then the patient does not have another appointment on the same day and in the time slot after *appt*.

3 The classes *Practice* and *PracTrans*

In this section we are going to complete a description of the implementation of the practice appointment system using the WorkPad. Now it would be usual to implement a database using specialist software. (A key difference between implementing the database using specialist software and doing so using the course code is that database software would preserve the data.) What is more, an application such as this appointment system would usually have a **GUI** (Graphical User Interface), designed as part of the software system, that would enable a user to execute required transactions. We do not have such facilities within the course code, so our implementation is necessarily a simplified version.

The implementation involves two further stages. The first of these involves a class **Practice**. This class enables communication between the classes **Member**, etc, introduced in Section 2. We then introduce a further class **PracTrans**. This has methods designed specifically to enable each of the required transactions discussed in Section 1. The specifications of these two classes are given in an Appendix at the end of the unit. These are large classes. For such classes, one is not going to remember all the details of the specification of each method; rather, one will use the specification as a reference when one needs to recall some detail. Before reading on, spend a few minutes looking at the specification of **Practice**. Note the following.

1. The first four attributes of **Practice** correspond to relations over each of the relation schemes introduced in the previous section. These have been implemented as vectors to enable adding and removing tuples.
2. The other two attributes give the number of days over which appointments can be made and the number of time slots in a day. These have been given default values of 5 and 8. (These numbers are small rather than realistic. This is to ensure that in our prototype implementation we do not need to create too much data.)
3. There are methods to add tuples to each of the relations (a new patient, a new member, a new appointment, or a new diary entry — this last is done by the method *setAct*), and methods to delete tuples.
4. There are methods to set details, such as whether an appointment has been kept.
5. There are methods to return information, such as: all the members, or whether a patient has a note to see a physiotherapist.
6. There are methods (*pat* and *mem*) that allow information to be transferred in a situation where a member is also a patient.
7. The methods *appointmentOK* and *patFree* correspond to predicates introduced in Section 2, and *findAppt* was also discussed there.
8. The parameters used to represent members and patients are sometimes identity codes and sometimes the objects themselves. (The choice we have made is essentially pragmatic.)

In several of the transactions that we described in Section 1, we want the possibility of a **null** member for certain classes. We do not have a **null** option in the course code. We have taken the pragmatic approach that the default values of the attributes of a class member correspond to a null option, and have chosen default values accordingly. Programmers often take such pragmatic decisions, but they should be carefully flagged.

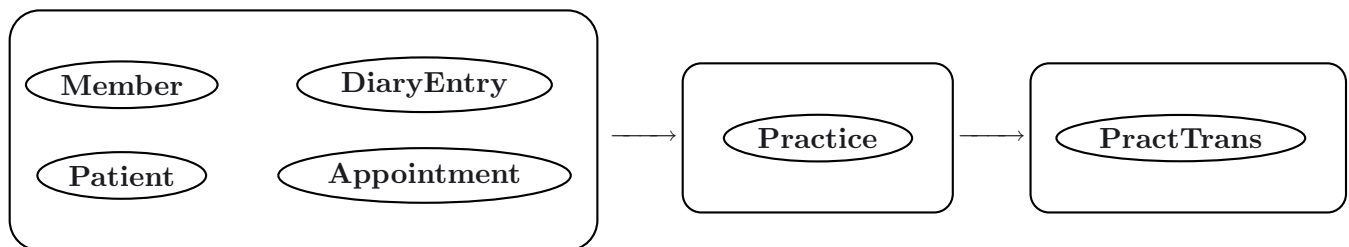
See page 42.

Decisions as to how to implement attributes of a class depend on what is available and what methods you anticipate will be needed. Such decisions usually take time and discussion that we have bypassed here.

In practice, references to objects will be passed as parameters rather than actual objects, so there would be little difference in efficiency.

The methods of **Practice** are intended to make the required transactions possible. Some transactions will require use of more than one of these methods. For example, the transaction of making an appointment requires the method of finding an appointment and then the method of adding it. It is not sensible to amalgamate these methods, since the complete transaction involves both a state change and a value being returned. Specialist software would make this transaction interactive, allowing offers of appointments to be refused until a satisfactory one was found. Such a system would also need to prevent other changes to the database while such offers are made.

Considerations such as these lead to the use of a three-layer architecture that is reflected even in our small prototype system. The classes corresponding to the four relation schemes, **MEMBER**, **PATIENT**, **APPOINTMENT** and **DIARYENTRY** form the bottom layer. The database itself consists of relations over these relation schemes. These are the main attributes of the **Practice** class, the middle layer of the architecture. The top layer contains the application processes that are directly accessible to the user. In our prototype, the top layer will be represented by a class **PracTrans**. This will have methods that carry out the transactions discussed on page 8.



If this design was being implemented in specialist database software it would be set up so that, when making an appointment say, the receptionist would start with a form that would ask for the patient's name. If several patients shared the same name the receptionist would ask for further information to select the correct patient. The patient's identity code would then be used in any subsequent transactions. This means that, when appropriate, we will have a precondition that a patient with the given identity code exists.

In general, the receptionist would be using forms to carry out the transactions. The receptionist would not directly access the **Practice** class or its methods. We will now introduce a class to help with simulating the transactions. Of course, we cannot simulate the use of forms in the WorkPad. These are replaced by methods of the new class, **PracTrans**.

We now turn to the class **PracTrans**. An object of type **PracTrans** has a single attribute, of type **Practice**. We call this attribute *local*, and in specifications of methods of **PracTrans**, *local* always means the single attribute of the receiving object. A full specification of **PracTrans** is given in the Appendix at the end of the unit, but there is no need to examine it now. In the remainder of this section we shall consider some of the methods of this class, and how such methods can be coded using the WorkPad (assuming that what we have available is the class **Practice**). We look first at methods that add a patient to the practice, and add a member. We first give their specifications.

method *addPatient*(*pName* in SeqOfChar) **state change**

pre true.

post A patient with name *pName* is added to *local*. The patient is allocated a unique patient identity code.

Use of this class design means that we can hide methods of **Practice** from an object of type **PracTrans**.

method *addMember*(*mName* in *SeqOfChar*, *type* in *Char*, *pat* in *Bool*) **state change**

pre *type* is 'D' or 'N' or 'P'.

post A member with name *mName* is added to *local*. The member is allocated a unique member identity code. The type of the member is *type*. If *pat* has value **true**, a patient is added to *local* and allocated a unique identity code. This code is the value of the attribute *patientID* of the member. If *pat* has value **false**, the attribute *patientID* of the member is the empty string.

A point to note here is that patients and members have identity codes from the class **mString**. However, to make the implementation easy to use, whenever a parameter of a method of **PracTrans** corresponds to an identity code we will use a parameter from the set *SeqOfChar*, which can then be entered as a literal sequence of characters in the WorkPad.

EXAMPLE 3.1

An implementation of *addPatient* is given below. This code first turns the input literal into an instance of **mString**, then calls the corresponding method of **Practice**.

```
method addPatientPracTrans(pName)
{
  var patID in mString
  patID.setString(pName)
  this.local.addPatient(patID)
}
```

Note that we give separate identifiers for the method and the implementation. The identifier of the implementation has *PracTrans* added at the end. In the WorkPad, the class implementation will need a statement *addPatient* **calls** *addPatientPracTrans*.

Activity 3.1

Give an implementation (*addMemberPracTrans*) of *addMember*.

Discussion An implementation is given below.

```
method addMemberPracTrans(mName,type,pat)
{
  var memID in mString
  memID.setString(mName)
  this.local.addMember(memID,type,pat)
}
```

Many of the methods needed for **PracTrans** are straightforward to implement, but some are not. We look next at making an appointment. In the first, informal, description of the appointment system we said that patients should be offered the first available appointment, which they could refuse. The transaction of making an appointment would consist of several steps. The receptionist would first find a suitable appointment and offer it to the patient. If the patient accepted the appointment, it is then added to the appointment list. If the patient did not accept the appointment then the next suitable appointment would be offered. This would continue until an appointment had been accepted and added to the list.

It is essential that *local* is unchanged until the last step of this process, and that the appointment added to the list is the same as the appointment offered. As a result, such a procedure should be regarded as a single transaction and we will implement it using a single method.

To do this, we need to specify a method of a type that we have avoided so far in this course. We need a method that will change the state of the receiving object (by adding an appointment to the list), and will also return a value — the appointment to be offered. To specify such a method, we will write the signature line as exemplified below.

method *makeAppointment*(*d* in **Int**, *t* in **Int**, *type* in **Char**, *pID* in *SeqOfChar*) **return** in **Appointment** **AND** state change

pre The time slot (*d*, *t*) is after the current day and time, and there is a patient with identity code *pID*, and *type* is 'D', 'N' or 'P'.

post If no suitable appointment can be found, then the returned value is the default value of **Appointment** and the state of *local* is not changed.

Otherwise, the returned value is *appt* where we have:

APPOINTMENTOK(*appt*), and *appt* occurs at or after the time slot (*d*, *t*) and the type of the member involved in *appt* is *type*.

Also, the state of *local* is changed as follows. The appointment *appt* is added to the appointment list; the diary is updated to show that the member involved in *appt* has an appointment at the time slot of *appt*; and, if *pID* is a member, the diary is updated to show that the member corresponding to *pID* has an appointment as a patient at the time slot of *appt*.

If the member involved in the appointment is a physiotherapist, an appointment is also added in the time slot following that of *appt* (i.e. (*d*, *t* + 1)), and the appropriate diary entries are added for the time slot (*d*, *t* + 1).

This is a complicated task, and to implement it, we will subdivide it. We will separate the cases where the patient is not a member of the practice and where the patient is a member. We will implement two subsidiary methods, one for each of these two cases. We look first at *makeAppointmentA*. This method has a specification that is identical to *makeAppointment*, except that the parameter *pID* is replaced by *patID* in **mString**, and has an additional precondition that *patID* is not a member of the practice.

An implementation of the method *makeAppointmentA* is given below in Example 3.2. Note that the method *findAppt* of **Practice** will return the default value of **Appointment** if no suitable appointment is found. The method *addAppt* will both add the appointment to the appointment list, and update the diary entry for the member concerned in the appointment. The specification of *addAppt* (given in the Appendix) shows that this also takes account of the situation where the appointment is with a physiotherapist.

EXAMPLE 3.2

```
method makeAppointmentAPracTrans(d,t,type,patID)
{
  var appt in Appointment
  appt <-- this.local.findAppt(d,t,type,patID)
  if (this.local.apptOK(appt)) then
  {
    this.local.addAppt(appt)
  }
  return appt
}
```

We will not implement the part of the process where the patient can refuse the appointment since this depends on a user interface that we do not have.

For a value *appt* returned by *findAppt*, *apptOK*(*appt*) will only be **false** if *appt* is **null** — see the postcondition of *findAppt* on page 44.

The method *makeAppointmentA* is not suitable if the patient is a member of the practice. In that case, we need to ensure that, when the appointment is made, they are entered in the diary as having an appointment as a patient at the appointment time. We now consider a method *makeAppointmentB* to make an appointment as a patient for a member of the practice. This has a specification identical to that of *makeAppointment*, except that the parameter *pID* is replaced by the member identifier *memID*, of type **mString**, and there is an additional precondition that *memID* is a member of the practice.

The test involving *apptOK* will ensure that the patient does not already have an appointment as a member at the proposed time

Activity 3.2

Think about the steps required in an implementation of *makeAppointmentB*. Attempt to code an implementation, but do not spend too long on this.

Discussion An implementation is given below.

```
method makeAppointmentBPracTrans(d,t,type,memID)
{
  var appt in Appointment
  var patID in mString    // the patient ID of memID
  var theMem in Member    // the member object for memID
  var appDay in Int
  var appTime in Int
  theMem <-- this.local.findMember(memID)
  patID <-- theMem.getPatientID()           //(1)
  appt <-- this.local.findAppt(d,t,type,patID)
  if (this.local.apptOK(appt)) then
  {
    this.local.addAppt(appt)
    appDay <-- appt.getDay()
    appTime <-- appt.getTime()
    this.local.setAct(appDay,appTime,memID,'P')   //(2)
    if (type == 'P') then
    {
      this.local.setAct(appDay,appTime+1,memID,'P') //(3)
    }
  }
  return appt
}
```

Remember that the attribute *action* of **DiaryEntry** is set as 'P' to show that a member has an appointment as a patient.

The value 'P' is also used for the attribute *memberType* of **Member** to show that a member is a physiotherapist.

Find a suitable appointment (using *findAppt*), and if this is not **null**, add it to the appointment list. The code for this is essentially the same as for *makeAppointmentA*, but we need to remember that the parameter for *findAppt* is a patient identifier. We find the identifier as a patient of the member *memID* in statement (1).

Also, we must update the diary. Statement (2) adds a diary entry for the member at the time of the appointment with activity set as 'P'. Also, if the appointment is with a physiotherapist, another entry for the next time slot is added, in statement (3).

To implement the method *makeAppointment* itself, we need first to determine whether or not *pID* is a member, then to use *makeAppointmentA* or *makeAppointmentB* as appropriate.

EXAMPLE 3.3

```
method makeAppointmentPracTrans(d,t,type,pID)
{
  var appt in Appointment
  var patID in mString    // pID as an mString
  var thePat in Patient   // the patient object
  var theMem in Member    // the member for pID
  var memID in mString    // the member ID of pID
  patID.setString(pID)
  thePat <-- this.local.findPatient(patID)
  theMem <-- this.local.mem(thePat)
  memID <-- theMem.getMemberID()
  if (theMem.getMemberType() == 'X') then          //(1)
  {
    appt <-- this.makeAppointmentA(d,t,type,patID)
  }
  else
  {
    appt <-- this.makeAppointmentB(d,t,type,memID)
  }
  return appt
}
```

To reflect the way we would use **PracTrans** to make appointments, only the method *makeAppointment* is given in the class specification. To prevent the direct use of *makeAppointmentA* and *makeAppointmentB*, most programming languages would allow one to designate them as **private**, but we do not have this facility in the *WorkPad*.

The condition in (1) will be **true** when *pID* is not a member.

We end this section by considering the following transaction.

Members can be declared unavailable for particular time slots.

This transaction requires some thought. It would not be appropriate to declare a member unavailable if they have an appointment, either as member or patient, at the given time. Hence this requirement requires feedback from the system. We will devise a method with two possible responses to a request to declare a member unavailable. One is that the member is already busy at the time in question; the other is that the diary is changed to declare the member unavailable. The method will change the state of *local* in the second case, and will also return a value, which we shall give as a string. The specification of this method, *makeBusy*, is given below. We consider its implementation in Exercise 3.1.

method *makeBusy*(*d* in Int, *t* in Int, *mID* in SeqOfChar) **return** in mString **AND** state change

pre The day *d* and time *t* are after the current day and time.
There is a member of *local* with identity code *mID*.

post If the member with identity code *mID* has a diary entry in which *act* has the value 'A', 'P' or 'U' at the time slot (*d*, *t*) then the returned value is "Already has an appointment", and the state of *local* is not changed.

If there is no such diary entry for *mID* then the diary is updated to include an entry for the member *mID* in the time slot (*d*, *t*) with the attribute *act* given the value 'U', and the returned value is "Declared unavailable".

Objectives for Section 3

After studying this section you should be able to do the following.

- Interpret the specifications of the classes **Practice** and **PracTrans**.
- Using **Practice**, give an implementation of a method of **PracTrans**.

Seen as reference documents.

To enable a required transaction.

Exercises on Section 3

Exercise 3.1

Give an implementation of *makeBusy*.

Exercise 3.2

- (a) Consider a method *cancelAppointmentA* that will cancel an appointment for a patient who is not a member of the practice. The parameters of the method are: the identity code of the patient concerned (*patID* of type **mString**); and the day *d* and time slot *t* of the appointment to be cancelled. Suggest a specification for *cancelAppointmentA*.
- (b) Give an implementation of *cancelAppointmentA* (as specified in the solution to part (a)). Note that **Practice** has a method *removeAppt* that will remove an appointment, which has an object of type **Appointment** as a parameter. The method *getPAppt* can be used to identify the appointment to be removed. The call *appt.getMemberID()* will give the identity code of the member involved in *appt*.

These parameters simulate an interface where the first step is to find the appointment to be cancelled.

Do not spend too long on this part, which is quite difficult.

4 Checking against the specification

In this section, we shall consider how to write code to test both that the methods of **PracTrans** enable us to perform the transactions identified in Section 1, and that the methods produce the desired results. We shall examine a carefully chosen selection of test cases. We shall go through each of the required transactions on page 8, check that **PracTrans** provides a method to carry out that transaction, and then devise code to test the method implementation.

This section contains a good deal of test code that should be run in the WorkPad. You may find it useful first to read through the section quickly, concentrating on the overall strategy of testing rather than on details of the code. Then go through again, running each code fragment in the WorkPad, and checking the results of the tests.

Activity 4.1

To test whether members and patients can be added to the practice, we can use the following code. Describe what you would look for in *members* and *patients* when this code fragment is executed.

Transaction 1 on page 8.

```
var trans in PracTrans
var members in Vector of Member
var patients in Vector of Patient
trans.addMember("Gabrys", 'P', true)
trans.addMember("Gabrys", 'P', false)
trans.addMember("Dil", 'D', true)
trans.addPatient("Dil")
trans.addPatient("Jordan")
members <-- trans.getMembers()
patients <-- trans.getPatients()
```

Discussion You would expect there to be three members: a doctor named Dil, and two physiotherapists, both named Gabrys, who should be distinguished from each other in some way. There should be four patients: called Gabrys, Dil, Dil and Jordan, where the second Dil is distinguished from the first one.

In each of the test cases described in this section, we leave you to check the code in the WorkPad, and see how the results compare with our expectations.

In testing the methods of **PracTrans**, it will be useful to have a method that will put some data into the database. This method does not reflect any transaction that is needed. It is something we include for convenience when testing. This method, *initPractice*, will add some members and patients to the practice. The method is specified below. (Recall that the attributes of a patient are: *patientID*, *patientInfo* and *physioNote*, while the attributes of a member are: *memberID*, *memberType*, *memberInfo* and *patientID*. For the purposes of our testing, what is stored under *patientInfo* and *memberInfo* is largely irrelevant. If a member is not a patient, then their *patientID* is stored as the **null** value — an empty string.)

method *initPractice()* **state change**

pre **true.**

post Some members and patients are added to *local*, as shown below.

Patients

```
("Vincent", "Vincent-1", false)
("Harding", "Harding-1", false)
("Jordan", "Jordan-1", false)
("Dil2", "Dil-2", false)
("Dil", "Dil-1", false)
("Lister", "Lister-1", false)
("Thomas", "Thomas-1", false)
("Gabrys", "Gabrys-1", false)
```

Members

```
("MDil", 'D', "Dil-1", "Dil")
("MMeehan", 'D', "Meehan-1", "")
("MLister", 'N', "Lister-1", "Lister")
("MThomas", 'D', "Thomas-1", "Thomas")
("MGabrys", 'P', "Gabrys-1", "Gabrys")
("MDil2", 'P', "Dil-2", "Dil2")
```

These examples illustrate how the WorkPad implementation stores the attributes of a member and a patient.

In our remaining tests, we will usually use *initPractice* to load this initial test data.

It is straightforward to check that the requirements about a note for the physiotherapist are met.

Activity 4.2

Write code to test the requirement that patients can be recorded as having or as not having a note for the physiotherapist.

Transaction 2 on page 8.

Discussion The following code checks that a patient can be declared to have a note, and that a patient can be declared not to have a note. After (1), *response* should be **true**, while after (2), it should be **false**.

```
var trans in PracTrans
var appt in Appointment
var response in Bool
trans.initPractice()
trans.setPhysioNote("Vincent",true)
response <-- trans.getPhysioNote("Vincent")    //(1)
trans.setPhysioNote("Vincent",false)
response <-- trans.getPhysioNote("Vincent")    //(2)
```

In all cases where we ask you to devise test code, your code will be different from that which we give. Try to compare your *strategy* with the text — not the details. Have you covered the same sort of situations?

The next transaction that we will check is:

Members can be made unavailable for particular time slots.

Transaction 3 on page 8.

The method *makeBusy* is intended to make this transaction possible, and testing this method requires some care. We can use the return value of *makeBusy* to check what has happened. We need to cover the possible reasons for getting the response “Already has an appointment”. The member might already have an appointment as a member, or as a patient. In addition, they might already have declared themselves unavailable. We also need to check what happens if an appointment with a physiotherapist is involved, since such appointments make members (and members as patients) unavailable for two time slots.

Activity 4.3

Devise some code to check the features discussed above.

Discussion The test code needs to make appointments of a patient with a member, of a member with another member (not a physiotherapist) and of a member with a physiotherapist. We then need to attempt to declare the members involved busy at the times covered by the appointments. We should include one attempt that we would expect to be successful. The following code does all these things. We have added the code for showing the practice diary so that we can check that it has been updated appropriately.

In the code below, note that whenever we enter a member as a sequence of characters in such test code (for example, as the first parameter in *makeBusy*), the value of *mID* starts with an ‘M’. The use of this character to distinguish members is illustrated by the postcondition of *initPractice*.

Where activities in this section ask you to devise test code, concentrate on the strategy for designing the code, rather than on getting the code that will actually run in the WorkPad. Sample test code to run in the WorkPad is provided.

In the test code below, we can anticipate that statement (1) will make an appointment with the doctor “MDil”, since (of the doctors provided by *initPractice*) Dil was added first.

```
var trans in PracTrans
var response in mString
trans.initPractice()
trans.makeAppointment(1,1,'D',"Vincent")      //(1)
trans.makeAppointment(1,2,'N',"Dil")          //(2)
trans.setPhysioNote("Thomas",true)
trans.makeAppointment(1,3,'P',"Thomas")        //(3)
response <-- trans.makeBusy(1,1,"MDil")        //(4)
response <-- trans.makeBusy(1,2,"MMeehan")      //(5)
response <-- trans.makeBusy(1,2,"MDil")        //(6)
response <-- trans.makeBusy(1,2,"MLister")      //(7)
response <-- trans.makeBusy(1,4,"MGabrys")      //(8)
response <-- trans.makeBusy(1,3,"MThomas")      //(9)
response <-- trans.showDiary()                  //(10)
```

The value of *response* should be:

after (4), (6), (7), (8) and (9): “Already has an appointment”

after (5): “Declared unavailable”.

After (10), *response* should show a diary with Dil busy at (1,1) and (1,2); Meehan busy at (1,2); Lister busy at (1,2); Thomas busy at (1,3) and (1,4); Gabrys busy at (1,3) and (1,4).

See the comment on the method *findAppt* in the Appendix .

In *initPractice*, the members were added in the order given in the postcondition.

The appointment made at (2) is bound to be with Lister — the only nurse.

The appointment made at (3) is with Gabrys, since that is the physiotherapist added first in *initPractice*. So Gabrys is also busy at time slot 4 — hence the predicted response after (8).

The crucial transactions involve the making of appointments and we will now check these. The two transactions involved are:

The next available appointment, at or after a given time slot and with a particular type of member, can be found. The appointment found will not break any of the requirements.

Transaction 4 on page 8.

An appointment can be added to the practice.

Transaction 5 on page 8.

In order to see what to check it will help to refer back to the predicate *APPOINTMENTOK*. This predicate had several parts, as below.

See Section 2.

1. The patient does not have another appointment on the same day and in the same time slot as the appointment.
2. The member does not have another appointment on the same day and in the same time slot as the appointment.
3. The member does not have another appointment, as patient, on the same day and in the same time slot as the appointment.
4. The member is not otherwise occupied on the same day and in the same time slot as the appointment.
5. The patient involved is not the member involved.
6. The appointment is not **null**.
7. If the member is a physiotherapist then the patient has a note for the physiotherapist.
8. If the member is a physiotherapist then the time slot has an odd number.
9. If the member is a physiotherapist then the time slot is not the last one in the day.
10. If the member is a physiotherapist then the patient does not have another appointment on the same day and in the time slot after the appointment.

11. If the member is a physiotherapist then the member does not have another appointment on the same day and in the time slot after the appointment.
12. If the member is a physiotherapist then the member does not have another appointment, as patient, on the same day and in the time slot after the appointment.
13. If the member is a physiotherapist then the member is not otherwise occupied on the same day and in the time slot after the appointment.

Before reading on, take a moment to consider what needs to be checked to ensure that the requirements for appointments are met.

Our test code needs to make the following checks.

- (a) There needs to be an appointment with each type of member.
- (b) We need to check that a member is not given an appointment at a time when they are busy.
- (c) We need to check that a member is not given two appointments for the same time.
- (d) We need to check that a patient is not given two appointments for the same time.
- (e) We need to check that a patient is not given an appointment with a physiotherapist if they do not have a note.
- (f) We need to check that a patient is given an appointment with a physiotherapist if they do have a note.
- (g) We need to check physiotherapy appointments with particular care to ensure that two consecutive appointments (on the same day) have been made. We need to check that the diary has been updated to show that both the physiotherapist, and the patient if a member, have been shown to have an appointment in both slots.
- (h) We need to check that a member cannot be given an appointment with themselves.

It is sensible in the first instance to devise several short pieces of code to check these.

Activity 4.4

For each of the cases (a)–(h) given above, devise some code to make the required check. (Your code will use the method *makeAppointment* to make appointments. The results can be checked by examining values returned by *showDiary* and *showApptList*.)

Discussion

- (a) Appointments with a physiotherapist are checked in (f), so we just check appointments with a doctor and a nurse here.

```
var trans in PracTrans
var response in mString
trans.initPractice()
trans.makeAppointment(1,1,'D',"Vincent")
trans.makeAppointment(1,2,'N',"Vincent")
response <-- trans.showApptList()
```


- (b) We need to check that a member is not given an appointment at a time when they are busy. The appointment made by the code below should not be with Dil.

```
var trans in PracTrans
var response in mString
trans.initPractice()
trans.makeBusy(1,2,"MDil")
trans.makeAppointment(1,2,'D',"Vincent")
response <-- trans.showApptList()
```

The method *findAppt* would return an appointment with “Dil” if available, since, of the doctors in the practice, Dil was added first.

- (c) We need to check that a member is not given two appointments for the same time. We will test this using the nurse, as there is only one nurse. The code below should make two appointments at different times.

```
var trans in PracTrans
var response in mString
trans.initPractice()
trans.makeAppointment(1,1,'N',"Vincent")
trans.makeAppointment(1,1,'N',"Jordan")
response <-- trans.showApptList()
```

- (d) We need to check that a patient is not given two appointments for the same time. Again, the code below should make two appointments at different times.

```
var trans in PracTrans
var response in mString
trans.initPractice()
trans.makeAppointment(1,1,'N',"Vincent")
trans.makeAppointment(1,1,'D',"Vincent")
response <-- trans.showApptList()
```

- (e) We need to check that a patient is not given an appointment with a physiotherapist if they do not have a note. So the code below should not make an appointment — the appointment list should be empty.

```
var trans in PracTrans
var response in mString
trans.initPractice()
trans.makeAppointment(1,1,'P',"Vincent")
response <-- trans.showApptList()
```

- (f) We need to check that a patient is given an appointment with a physiotherapist if they do have a note.

```
var trans in PracTrans
var response in mString
trans.initPractice()
trans.setPhysioNote("Dil",true)
trans.makeAppointment(1,3,'P',"Dil")
response <-- trans.showApptList()
```

- (g) We need to check physiotherapy appointments with particular care to ensure that two consecutive appointments on the same day are made. We also need to check that the diary has been updated to show that the physiotherapist and the patient, if a member, have been shown to have an appointment in both slots. Thus in this test the patient will be a member of the practice and we will check the diary as well as the appointment list. Because there are only eight time slots in a day, the code below should make the appointment on day 2. The diary should show both Dil and the physiotherapist with actions at the two appointment times.

```
var trans in PracTrans
var response in mString
var response1 in mString
trans.initPractice()
trans.setPhysioNote("Dil",true)
trans.makeAppointment(1,8,'P',"Dil")
response <-- trans.showApptList()
response1 <-- trans.showDiary()
```

- (h) We need to check that a member cannot be given an appointment with themselves.

We will run two tests here. First, since there is only one nurse (Lister), the nurse should not be able to get an appointment with a nurse!

```
var trans in PracTrans
var response in mString
trans.initPractice()
trans.makeAppointment(1,1,'N',"Lister")
response <-- trans.showApptList()
```

We will also check that Dil (as a patient) is allocated a different doctor.

```
var trans in PracTrans
var response in mString
trans.initPractice()
trans.makeAppointment(1,1,'D',"Dil")
response <-- trans.showApptList()
```

Although we have written test code for each of our listed checks, there is the possibility that the various pieces of code interfere with each other. To check for this systematically would be tricky, but we can run a piece of code making various appointments, and check for clashes.

Activity 4.5

What features would you look for after running the code fragment below?

```
var trans in PracTrans
var response in mString
var response1 in mString
trans.initPractice()
trans.setPhysioNote("Thomas",true)
trans.makeAppointment(1,1,'D',"Thomas")
trans.makeAppointment(1,1,'D',"Harding")
trans.makeAppointment(1,1,'D',"Jordan")
trans.makeAppointment(1,1,'P',"Thomas")
trans.makeAppointment(1,2,'N',"Dil")
trans.makeAppointment(1,1,'D',"Dil")
response <-- trans.showApptList()
response1 <-- trans.showDiary()
```

Discussion There should be appointments with a doctor for the patients Thomas, Harding and Jordan. These should all involve either different times or different doctors. Thomas should have an appointment with a physiotherapist, which is not at the same time as her appointment with a doctor, and should cover two successive time slots. Dil should have appointment with a nurse, and with a doctor, that do not clash with each other, nor with any appointment he has as a member.

Activity 4.6

Write code to check that the transactions in (a) and (b) below can be carried out. (You may need to check the specification of **PracTrans** to determine which methods enable these transactions.)

(a) Medical notes about an appointment can be added.

Transaction 8 on page 8.

(b) A record of whether an appointment has been kept can be added.

Transaction 9 on page 8.

Discussion The method *setNotes* enables notes to be added, and *setKept* enables a record of whether the appointment is kept to be added. In both cases the appointment has to exist before the required details can be added. The code below tests these methods. Running this should result in the appropriate entries for *kept* and *notes* in the single appointment.

```
var trans in PracTrans
var appt in Appointment
var response in mString
trans.initPractice()
trans.setPhysioNote("Harding",true)
appt <-- trans.makeAppointment(2,3,'P',"Harding")
trans.setKept(appt,true)
trans.setNotes(appt,"test")
response <-- trans.showApptList()
```

We have made an appointment with a physiotherapist to check what is shown for the appointment in the following time slot.

We now move on to the transaction:

An appointment can be cancelled.

Transaction 10 on page 8.

To perform this, we use the method *cancelAppointment* of **PracTrans**, whose specification is given below.

method *cancelAppointment*(*pID* in *SeqOfChar*, *d* in *Int*, *t* in *Int*) **state change**

pre The time slot (*d*, *t*) is after the current day and time.

There is an appointment (say *appt*) for the patient with identity code *pID* at the time slot (*d*, *t*).

If the member involved in *appt* is a physiotherapist, then *appt* is the first appointment of the pair.

post The state of *local* is changed as follows. The appointment *appt* is deleted from the appointment list (so that there is no appointment for the patient with identity code *pID* at the time slot (*d*, *t*)). If the member involved in *appt* is a physiotherapist, then the subsequent appointment is also deleted, so that there is also no appointment for the patient with identity code *pID* in the time slot (*d*, *t* + 1). The corresponding diary entry or entries for the member involved in the appointment have the value of the attribute *act* changed to 'F'. If the patient is a member of the practice, then the corresponding diary entries for the patient as member are modified to set *act* to 'F'.

In Exercise 3.2, we considered a method *cancelAppointmentA*. That cancels an appointment for a patient who is not a member. Just as with making an appointment, we also need to deal with the case where the patient is a member. To write suitable code, we can first deal with that case, using a method *cancelAppointmentB*. In the visible methods of **PracTrans**, we only need a combined method, as specified above, that will cancel an appointment whether or not the patient is a member. (We omit the remaining details of the implementation of this method.)

Activity 4.7

Write code to test the method *cancelAppointment*.

Discussion We test the two cases separately. First, we look at patients who are not members. To test this case, we will make two appointments and then cancel them. One of the appointments should be with a physiotherapist, and to make that we need to ensure the patient has a note.

```
var trans in PracTrans
var response in mString
var response1 in mString
trans.initPractice()
trans.makeAppointment(2,3,'D',"Harding")
trans.setPhysioNote("Jordan",true)
trans.makeAppointment(2,3,'P',"Jordan")
response <-- trans.showApptList()
response1 <-- trans.showDiary()           //(1)
trans.cancelAppointment("Harding",2,3)
trans.cancelAppointment("Jordan",2,3)
response <-- trans.showApptList()
response1 <-- trans.showDiary()           //(2)
```

Cancelling an appointment requires both that it is removed from the list and also that the member involved in the appointment is declared available, so we examine both the appointment list and the diary. After (1), the appointments should have been made, and after (2), they should have been deleted — with the appropriate adjustments to the diary in each case.

We now test appointments for patients who are members of the practice. Again, check that the appointments are made after (1), and cancelled after (2), with the appropriate adjustments to the diary in each case.

```
var trans in PracTrans
var response in mString
var response1 in mString
trans.initPractice()
trans.makeAppointment(2,3,'D',"Dil")
trans.setPhysioNote("Gabrys",true)
trans.makeAppointment(2,3,'P',"Gabrys")
response <-- trans.showApptList()
response1 <-- trans.showDiary()           //(1)
trans.cancelAppointment("Dil",2,3)
trans.cancelAppointment("Gabrys",2,3)
response <-- trans.showApptList()
response1 <-- trans.showDiary()           //(2)
```

Activity 4.8

Devise code to check the transactions in (a)–(d) below.

- | | |
|--|---------------------------|
| (a) A list of appointments for a member can be returned. | Transaction 6 on page 8. |
| (b) A list of appointments for a patient can be returned. | Transaction 7 on page 8. |
| (c) Details of an appointment for a given time and patient can be found. | Transaction 11 on page 8. |
| (d) Details of an appointment for a given time and member can be found. | Transaction 12 on page 8. |

Discussion We start the code by making a variety of appointments.

```
var trans in PracTrans
var appt in Appointment
var response in mString
trans.initPractice()
trans.makeAppointment(1,1,'D',"Harding")           //(1)
appt <-- trans.getPAppt("Harding",1,1)             //(2)
trans.setKept(appt,true)                           //(3)
trans.makeAppointment(1,2,'N',"Vincent")           //(4)
trans.setPhysioNote("Harding",true)
trans.makeAppointment(1,3,'P',"Harding")
trans.makeAppointment(1,2,'N',"Dil")               //(5)
trans.makeAppointment(2,1,'N',"Meehan")
trans.cancelAppointment("Meehan",2,1)
response <-- trans.getMappts("MDil")                //(6)
response <-- trans.getPAppts("Harding")
appt <-- trans.getAppt("MLister",1,2)               //(7)
```

It is sensible to include some cancellations. We then call the appropriate methods to check each of these four transactions: for (a), the method *getMAppts*; for (b), *getPAppts*; for (c), *getPAppt*; for (d), *getAppt*.

If we examine *response* after statement (6), we expect to see any appointments involving Dil as a member.

If we examine *appt* after statement (6), we expect to see details of the appointment object relating to Harding's appointment at *day* = 1, *time* = 1, which is set up by (1) and modified by (3).

If we examine *response* after statement (7), we expect to see any appointments involving Harding as a patient.

If we examine *appt* after statement (7), we expect to see details of the appointment object relating to the member Lister's appointment at *day* = 1, *time* = 2. This will be created at (4), since Lister is the only nurse.

We have now been through the list of transactions and checked that each one can be executed, and that our tests produce the correct results. (Transaction 13 is satisfied by the method *showApptList*, and 14 is satisfied by *showDiary*, and each of these has been tested.) The tests we have carried out have been reasonably systematic but cannot cover all possibilities. It is important to realise that testing is not the same as proving that system is correct, even the use of carefully designed tests covering a selection of typical cases.

Systematically testing each transaction is valuable, but it is advisable also to look out for, and to test, awkward cases. By awkward cases, we do not mean incorrect inputs or arguments of functions that do not satisfy the preconditions; we simply mean things like dealing correctly with the first or last element of an array or, where a precondition specifies that an argument must lie within a given range, dealing with the end points of the range.

Such testing would not be adequate for a safety critical system.

Activity 4.9

Can you think of any "awkward cases" that we have not tested yet? If you can, devise suitable test code.

Discussion You may have thought of checking the following cases.

1. If we try to start an appointment with a physiotherapist on an even time slot, it starts at the first available odd time slot.
2. If we try to make an appointment when there are no more time slots left for the day concerned, an appointment is made for the next day.
3. No problems are caused by appointments being made out of order.

Suitable code to test these features is given on the following page. As there is only one nurse, and there are only eight time slots in a day, the appointment made for Vincent by (1) should in day 2 (or later). The appointment made for Harding at (2) should be in day 2 or later, since Harding has already made an appointment on day 1, time 8. The appointment made by (3) is out of time sequence, to test the third point above. It also tests the second point, by trying to start an appointment with a physiotherapist at *time* = 4.

You may well have thought of something different.

We checked in Activity 4.4(g) that an appointment with a physiotherapist will not be split over two days.

```

var trans in PracTrans
var appt in Appointment
var response in mString
trans.initPractice()
trans.makeAppointment(1,8,'N',"Harding")
trans.makeAppointment(1,8,'N',"Vincent")    //(1)
trans.setPhysioNote("Harding",true)
trans.setPhysioNote("Dil",true)
trans.makeAppointment(1,7,'P',"Harding")    //(2)
trans.makeAppointment(1,4,'P',"Dil")        //(3)
response <-- trans.showApptList()

```

The testing we have carried out should reassure you that the system is reasonable and satisfies the requirements. The approach to testing we have used shows the value of making requirements reasonably formal, and having clear specifications of the classes with which we are working.

Objectives for Section 4

After studying this section you should be able to do the following.

- Decide on tests to verify that a transaction has been correctly implemented, and interpret the results of such tests.

Exercises on Section 4

Exercise 4.1

Suppose that we wish to add the additional requirement below to the appointment software. We will satisfy this by extending the class **PracTrans**.

A patient who has no appointments can be removed from the practice.

- (a) Give an implementation (*hasApptsPracTransPlus*) of the method *hasAppts* specified below.

method *hasAppts*(*pID* in *SeqOfChar*) **return** in **Bool**

pre *pID* is not a member.

post The returned value is **true** if the patient with ID code *pID* has one or more appointments, and is **false** otherwise.

This exercise deals only with a patient who is not a member.

- (b) Specify a method *removePat* that has a parameter *pID*, which is a sequence of characters giving a patient's ID code. This should return a value of type **mString**, which is "patient deleted" if the patient *pID* has no appointments, and which is "patient has appointments" otherwise. In the first case, the method should also change the state of *local* by removing the patient *pID* from the practice
- (c) Give an implementation (*removePatPracTransPlus*) of *removePat*.
- (d) Give code that declares a class **PracTransPlus** which extends the class **PracTrans** by adding the two methods *hasAppts* and *removePat*.
- (e) Give test code that could be used to test your extended class.

Solutions to the exercises

Solution 1.1

(a) Since the question states that this is a proposition, the solution really should involve only bound variables. (You may well not have included the binding, in which case you will have a predicate.) We read the given statement as an assertion that should hold for all suitable values of $aPat$, etc. To express “ t is odd” we can use the type function MOD of **Int**. An integer is odd if, when it is divided by 2, the remainder is 1.

$$\forall aPat \text{ in } Patient [\forall aMem \text{ in } Member [\forall t \text{ in } TimeSlot [\forall d \text{ in } Day [\\ (APPOINTMENT(aMem, aPat, d, t) \wedge ISPHYSIO(aMem) \wedge (MOD(t, 2) == 1)) \\ \Rightarrow APPOINTMENT(aMem, aPat, d, t + 1)]]]]]$$

(b) $\exists aName \text{ in } Member [ISNURSE(aName) \wedge \exists d \text{ in } Day \\ \neg(\exists t \text{ in } TimeSlot [\exists aName1 \text{ in } Patient [APPOINTMENT(aName, aName1, d, t)])]]]$

Solution 2.1

The method *setKept* will require two parameters: one in **Appointment** and one in **Bool** (the appointment we wish to adjust, and the value we wish to give to its attribute *kept*).

Solution 2.2

$$ISPHYSIO(findMember(appt.getMemberID())) \\ \Rightarrow PATFREE(appt.getDay(), appt.getTime()+1, findPatient(appt.getPatientID()))$$

Solution 3.1

A suitable implementation is given below, followed by some comments on the code.

```
method makeBusyPracTrans(d,t,mID)
{
  var response in mString
  var action in Char
  var memID in mString
  memID.setString(mID)                                //(1)
  action <-- this.local.getAct(d,t,memID)              //(2)
  if ((action=='A')\/(action=='P')\/(action=='U')) then //(3)
  {
    response.setString("Already has an appointment")
  }
  else
  {
    this.local.setAct(d,t,memID,'U')
    response.setString("Declared unavailable")
  }
  return response
}
```

Statement (1) sets *memID* to the *mString* corresponding to the string of characters input as *mID*. (2) uses the method *getAct* of **Practice** to find the activity (if any) currently recorded in the diary.

We then check whether the member is busy at the given time (3), and then set *response* to the appropriate string value to be returned, and use the method *setAct* of **Practice** to add a new diary entry if they are not busy (or to modify the existing diary entry if they are currently recorded as ‘F’).

Solution 3.2

(a) A specification is given below. You are unlikely to have the same details! In particular, the precondition contains features that are design decisions, and you could well have taken a different approach. We have required that the day and time are after the current day and time, and that there is an appointment for the specified patient at the day and time given. These preconditions seem advisable. The question specified that *patID* is not a member of the practice. We have included a further precondition, that if the appointment is with a physiotherapist then it should be the first appointment of the pair. (This will remove the need for a good deal of checking in the implementing code.) The postcondition needs to deal with the case where the appointment is with a physiotherapist.

method *cancelAppointmentA*(*patID* in mString, *d* in Int, *t* in Int) **state change**

pre *d* and *t* give a time slot after the current day and time.

The patient with identity code *patID* is not a member of the practice.

There is an appointment (say *appt*) for the patient with identity code *patID* at the time slot (*d*, *t*).

If the member involved in *appt* is a physiotherapist, then *appt* is the first appointment of the pair.

post The state of *local* is changed as follows. The appointment *appt* is deleted from the appointment list (so that there is no appointment for the patient with identity code *patID* at the time slot (*d*, *t*)). If the member involved in *appt* is a physiotherapist, then there is also no appointment for the patient with identity code *patID* in the time slot (*d*, *t* + 1). The corresponding diary entry or entries for the member involved in the appointment have had the value of *act* changed to 'F'.

(b) An implementation is given below.

```
method cancelAppointmentAPracTrans(patID,d,t)
{
  var appt in Appointment      // the appointment to be deleted
  var appt1 in Appointment     // the next appointment
  var mem in Member            // the member involved
  var memID in mString         // the member's ID
  appt <-- this.local.getPAppt(patID,d,t)
  this.local.removeAppt(appt)  // delete appt
  memID <-- appt.getMemberID() // the ID of the member
  mem <-- this.local.findMember(memID) // the member object
  if (mem.getMemberType() == 'P') then
  {
    // if the member is a physio then
    appt1 <-- this.local.getPAppt(patID,d,t+1)
    this.local.removeAppt(appt1)
  }
  // delete the next appointment
}
```

Solution 4.1

(a) We can use the method *getPAppts* to check whether the patient has any appointments. This returns an instance of **mString**, which will be empty if there are no appointments. Remember that we use the method *equal* of **mString** to test whether two strings are equal. The default value of **mString** is an empty string, so *temp* will be empty after statement (1) in the code below.

```
method hasApptsPracTransPlus(pID)
{
  var b in Bool
  var temp in mString                                //(1)
  if (temp.equal(this.getPAppts(pID))) then
  {
    b <-- false
  }
  else
  {
    b <-- true
  }
  return b
}
```

(b) A specification of *removePat* is given below.

method *removePat*(*pID* in SeqOfChar) **return** in mString **AND** state change
pre There is a patient with ID code *pID*.

post This returned value is “patient deleted” if the patient with identifier *pID* has no appointments, and is “patient has appointments” otherwise. In the first case, the state of *local* is changed by removing the patient with identifier *pID* from the practice.

(c) Suitable code is given below. Since this uses *hasAppts*, we will need to be careful to put the implementation of *hasAppts* first when entering this code into the WorkPad.

```
method removePatPracTransPlus(pID)
{
  var s in mString
  var patID in mString
  patID.setString(pID)
  if (this.hasAppts(pID)) then
  {
    s.setString("patient has appointments")
  }
  else
  {
    this.local.removePatient(patID)
    s.setString("patient deleted")
  }
  return s
}
```

(d) We need to give the two method implementations from (a) and (c), in that order, preceded by the code below.

```

class PracTransPlus

{
  this.hasAppts calls hasApptsPracTransPlus
  this.removePat calls removePatPracTransPlus
}

PracTransPlus extends PracTrans

```

(e) Suitable test code is given below. After (1), *response* should be **false**, and *response1* should be “patient deleted”. After (3), because Vincent is given an appointment at (2), *response* should be **true**, and *response1* should be “patient has appointments”. After (4), *v* should show that Jordan has been removed as a patient, but Vincent has not been removed.

```

var trans in PracTransPlus
var response in Bool
var response1 in mString
var v in Vector of Patient
trans.initPractice()
response <-- trans.hasAppts("Jordan")
response1 <-- trans.removePat("Jordan")           //(1)
trans.makeAppointment(2,3,'D',"Vincent")         //(2)
response <-- trans.hasAppts("Vincent")
response1 <-- trans.removePat("Vincent")          //(3)
v <-- trans.getPatients()                         //(4)

```

Appendix: Class specifications

The class Practice

Attributes

members in Vector of Member
patients in Vector of Patient
appts in Vector of Appointment
diary in Vector of DiaryEntry
days in Int
times in Int

Default

days = 5; *times* = 8; *members* = []; *patients* = []; *appts* = []; *diary* = []

Method signatures

addPatient(*patName* in mString) state change
addMember(*memName* in mString, *type* in Char, *pat* in Bool) state change
setAct(*d* in Int, *t* in Int, *memID* in mString, *act* in Char) state change
removePatient(*patID* in mString) state change
removeMember(*memID* in mString) state change

setDays(*d* in Int) state change
setTimes(*t* in Int) state change

These methods set the value of an attribute of practice.

setPhysioNote(*patID* in mString, *aNote* in Bool) state change
setKept(*appt* in Appointment, *isKept* in Bool) state change
setNotes(*appt* in Appointment, *n* in mString) state change

getMembers() return in Vector of Member
getPatients() return in Vector of Patient
getAppts() return in Vector of Appointment
getDiary() return in Vector of DiaryEntry
getDays() return in Int
getTimes() return in Int

Each of these methods returns the current value of an attribute of **Practice**.

getAct(*d* in Int, *t* in Int, *memID* in mString) return in Char
getPhysioNote(*patID* in mString) return in Bool
getAppt(*memID* in mString, *d* in Int, *t* in Int) return in Appointment
getPAppt(*patID* in mString, *d* in Int, *t* in Int) return in Appointment
getMAppts(*memID* in mString) return in Vector of Appointment
getPAppts(*patID* in mString) return in Vector of Appointment

findMember(*memID* in mString) return in Member
findPatient(*patID* in mString) return in Patient
pat(*aMem* in Member) return in Patient
mem(*aPat* in Patient) return in Member

apptOK(*appt* in Appointment) return in Bool
patFree(*d* in Int, *t* in Int, *patID* in mString) return in Bool
findAppt(*d* in Int, *t* in Int, *type* in Char, *patID* in mString) return in Appointment
addAppt(*appt* in Appointment) state change
removeAppt(*appt* in Appointment) state change

Method specifications

We omit the specifications of the methods that get and set values of the attributes of **Practice**, which are routine.

method *addPatient*(*patName* in **mString**) **state change**

pre **true**.

post A patient with name *patName* is added to the practice. The patient is allocated a unique patient identity code.

method *addMember*(*memName* in **mString**, *type* in **Char**, *pat* in **Bool**) **state change**

pre *type* in {'D', 'N', 'P'}.

post A member with name *memName* is added to the practice and allocated a unique identity code. The type of the member is *type*. If *pat* has value **true**, a patient is added to the practice and allocated a unique identity code. This code is the value of the attribute *patientID* of the member. If *pat* has value **false**, the attribute *patientID* of the member is an empty string.

method *setAct*(*d* in **Int**, *t* in **Int**, *memID* in **mString**, *act* in **Char**) **state change**

pre There is a member of the practice with identity code *memID*.

act is 'A', 'U', 'P' or 'F'.

post An entry (*d*, *t*, *memID*, *act*) is added to the diary.

method *removePatient*(*patID* in **mString**) **state change**

pre The patient with identity code *patID* has no appointments.

post The patient with identity code *patID* is no longer listed as a patient.

method *removeMember*(*memID* in **mString**) **state change**

pre The member with identity code *memID* has no appointments as a member.

post The member with identity code *memID* is no longer listed as a member.

method *setPhysioNote*(*patID* in **mString**, *aNote* in **Bool**) **state change**

pre There is a patient with identity code *patID*.

post The attribute *physioNote* of the patient with identity code *patID* is set to the value *aNote*.

method *setKept*(*appt* in **Appointment**, *isKept* in **Bool**) **state change**

pre *appt* is listed as an appointment.

If *appt* is with a physiotherapist, then it is the first appointment of the pair.

post The attribute *kept* of *appt* is changed to the value *isKept*.

If the appointment is with a physiotherapist, then the same change is made to the appointment in the following time slot.

method *setNotes*(*appt* in **Appointment**, *n* in **mString**) **state change**

pre *appt* is listed as an appointment.

If *appt* is with a physiotherapist, then it is the first appointment of the pair.

post The attribute *notes* of *appt* is changed to the value *n*.

If the appointment is with a physiotherapist, then the same change is made to the appointment in the following time slot.

method *getAct*(*d* in **Int**, *t* in **Int**, *memID* in **mString**) **return in Char**

pre There is a member with identity code *memID*.

post The returned value is the value of the attribute *act* in the diary entry for the member with identity code *memID* at day *d* and time slot *t*. If there is no such diary entry then the returned value is 'F'.

method *getPhysioNote*(*patID* in **mString**) **return in Bool**

pre There is a patient with identity code *patID*.

post The returned value is the value of the attribute *physioNote* of the patient with identity code *patID*.

method *getAppt*(*memID* in **mString**, *d* in **Int**, *t* in **Int**) **return in Appointment**

pre There is a member with identity code *memID*.

post The returned value is the appointment for the day *d* and time slot *t* for the member with identity code *memID*. (If there is no such appointment then a **null** appointment is returned.)

method *getPAppt*(*patID* in **mString**, *d* in **Int**, *t* in **Int**) **return in Appointment**

pre There is a patient with identity code *patID*.

post The returned value is the appointment for the day *d* and time slot *t* for the patient with identity code *patID*. (If there is no such appointment then a **null** appointment is returned.)

method *getMAppts*(*memID* in **mString**) **return in Vector of Appointment**

pre There is a member with identity code *memID*.

post The returned value is a vector containing all the appointments for the member with identity code *memID*.

method *getPAppts*(*patID* in **mString**) **return in Vector of Appointment**

pre There is a patient with identity code *patID*.

post The returned value is a vector containing all the appointments for the patient with identity code *patID*.

method *findMember*(*memID* in **mString**) **return in Member**

pre There is a member of the practice with identity code *memID*.

post The returned value is the member with identity code *memID*.

method *findPatient*(*patID* in **mString**) **return in Patient**

pre There is a patient of the practice with identity code *patID*.

post The returned value is the patient with identity code *patID*.

method *pat*(*aMem* in **Member**) **return in Patient**

pre **true**.

post If *aMem* is a patient then the returned value is the patient with the same patient identity code as the member; otherwise the returned value is the default patient value, which is **null**.

method *mem*(*aPat* in **Patient**) **return in Member**

pre **true**.

post If *aPat* is a member then the returned value is the member with the same patient identity code as the patient; otherwise the returned value is the default member value, which is **null**.

method *apptOK*(*appt* in **Appointment**) **return in Bool**

pre **true**.

post The return value is *APPOINTMENTOK*(*appt*).

The conditions combined to form *APPOINTMENTOK* are given on pages 15 and 16.

method *patFree*(*d* in **Int**, *t* in **Int**, *patID* in **mString**) **return in Bool**

pre There is a patient of the practice with the identity code *patID*, say *pat*.

post The returned value is *PATFREE*(*d*, *t*, *pat*).

See page 16 for the definition of *PATFREE*.

method *findAppt*(*d* in **Int**, *t* in **Int**, *type* in **Char**, *patID* in **mString**) **return in Appointment**

pre (*d* > 0) ∧ (*t* > 0) and there is a patient of the practice with identity code *patID* and *type* is 'D', 'N' or 'P'.

post If such an appointment can be found, the returned value is *appt* where *APPOINTMENTOK*(*appt*), and *appt* occurs at or after the time slot (*d*, *t*), and the type of the member involved in *appt* is *type*.

Otherwise the returned value is the default value of **Appointment**.

If there is more than one appointment satisfying the required condition, then the value of *appt* is chosen as follows. Take the first available time slot, and, if there is more than one member of the correct type available, then take the member who was added to the practice first.

method *addAppt*(*appt* in **Appointment**) **state change**

pre *APPOINTMENTOK*(*appt*).

post The appointment *appt* is added to the appointment list, and the diary is updated to show that the member involved in *appt* has an appointment at the time of *appt*, and if the member involved in *appt* is a physiotherapist then the corresponding changes are made for the time slot following that of *appt*.

method *removeAppt*(*appt* in **Appointment**) **state change**

pre *appt* is listed as an appointment.

post The appointment *appt* is removed from the appointment list. The diary is changed to show that the member involved in *appt* is free at the time of *appt*.

This same comment applies to *addAppointment*, and to the method *makeAppointment* of **PracTrans**.

The class PracTrans

Attributes *local* in **Practice**

Default *local* is in the default state of **Practice**

Method signatures

addPatient(*pName* in *SeqOfChar*) **state change**

addMember(*mName* in *SeqOfChar*), *type* in *Char*, *pat* in *Bool*) **state change**

findPatients(*pName* in *SeqOfChar*) **return in Vector of Patient**

findMembers(*mName* in *SeqOfChar*) **return in Vector of Member**

getPatients() **return in Vector of Patient**

getMembers() **return in Vector of Member**

setPhysioNote(*pID* in *SeqOfChar*, *note* in *Bool*) **state change**

getPhysioNote(*pID* in *SeqOfChar*) **return in Bool**

makeBusy(*d* in *Int*, *t* in *Int*, *mID* in *SeqOfChar*) **return in mString AND state change**

makeAppointment(*d* in *Int*, *t* in *Int*, *type* in *Char*, *pID* in *SeqOfChar*)

return in Appointment AND state change

cancelAppointment(*pID* in *SeqOfChar*, *d* in *Int*, *t* in *Int*) **state change**

getAppt(*mID* in *SeqOfChar*, *d* in *Int*, *t* in *Int*) **return in Appointment**

getPAppt(*pID* in *SeqOfChar*, *d* in *Int*, *t* in *Int*) **return in Appointment**

getMAppts(*mID* in *SeqOfChar*) **return in mString**

getPAppts(*pID* in *SeqOfChar*) **return in mString**

setKept(*appt* in **Appointment**, *isKept* in *Bool*) **state change**

setNotes(*appt* in **Appointment**, *n* in *SeqOfChar*) **state change**

showDiary() **return in mString**

showApptList() **return in mString**

getAct(*d* in *Int*, *t* in *Int*, *mID* in *SeqOfChar*) **return in Char**

initPractice() **state change**

Method specifications

method *addPatient*(*pName* in *SeqOfChar*) **state change**

pre *true*.

post A patient with name *pName* is added to *local*. The patient is allocated a unique patient identity code.

In all these specifications, *local* refers to the attribute *local* of the receiving object.

method *addMember*(*mName* in SeqOfChar, *type* in Char, *pat* in Bool) **state change**

pre *type* is 'D' or 'N' or 'P'.

post A member with name *mName* is added to *local*. The member is allocated a unique member identity code. The type of the member is *type*. If *pat* has value **true**, a patient is added to *local* and allocated a unique identity code. This code is the value of the attribute *patientID* of the member. If *pat* has value **false**, the attribute *patientID* of the member is the empty string.

method *findPatients*(*pName* in SeqOfChar) **return in Vector of Patient**

pre **true**.

post The returned value is a vector containing all the patients of *local* with name *pName*.

method *findMembers*(*mName* in SeqOfChar) **return in Vector of Member**

pre **true**.

post The returned value is a vector containing all the members of *local* with name *mName*.

method *getPatients*() **return in Vector of Patient**

pre **true**.

post The returned value is a vector containing all the patients of *local*.

method *getMembers*() **return in Vector of Member**

pre **true**.

post The returned value is a vector containing the all the members of *local*.

method *setPhysioNote*(*pID* in SeqOfChar, *note* in Bool) **state change**

pre There is a patient with identity code *pID*.

post In *local*, the attribute *physioNote* of the patient with identity code *pID* is set to the value *note*.

method *getPhysioNote*(*pID* in SeqOfChar) **return in Bool**

pre There is a patient with identity code *pID*.

post The returned value is the value of the attribute *physioNote* of the patient with identity code *pID*.

method *makeBusy*(*d* in Int, *t* in Int, *mID* in SeqOfChar) **return in mString AND state change**

pre The time slot (*d*, *t*) is after the current day and time.

There is a member of *local* with identity code *mID*.

post If the member with identity code *mID* has a diary entry in which *act* has the value 'A', 'P' or 'U' at the time slot (*d*, *t*) then the returned value is "Already has an appointment", and the state of *local* is not changed.

If there is no such diary entry for *mID* then the diary is updated to include an entry for the member *mID* in the time slot (*d*, *t*) with the attribute *act* given the value 'U', and the returned value is "Declared unavailable" .

method *makeAppointment*(*d* in Int, *t* in Int, *type* in Char, *pID* in SeqOfChar) **return in Appointment AND state change**

pre The time slot (*d*, *t*) is after the current day and time,

and there is a patient with identity code *pID*,

and *type* is 'D', 'N' or 'P'.

post If no suitable appointment can be found, then the returned value is the default value of **Appointment** and the state of *local* is not changed.

Otherwise, the returned value is *appt* where we have:

APPOINTMENTOK(*appt*), and *appt* occurs at or after the time slot (*d*, *t*) and the type of the member involved in *appt* is *type*.

Also, the state of *local* is changed as follows.

The appointment *appt* is added to the appointment list; the diary is updated to show that the member involved in *appt* has an appointment at the time slot of *appt*; and, if *pID* is a member, the diary is updated to show that the member corresponding to *pID* has an appointment as a patient at the time slot of *appt*.

If the member involved in the appointment is a physiotherapist, an appointment is also added in the time slot following that of *appt* (i.e. $(d, t + 1)$), and the appropriate diary entries are added for the time slot $(d, t + 1)$.

method *cancelAppointment*(*pID* in SeqOfChar, *d* in Int, *t* in Int) **state change**

pre The time slot (d, t) is after the current day and time.

There is an appointment (say *appt*) for the patient with identity code *pID* at the time slot (d, t) .

If the member involved in *appt* is a physiotherapist, then *appt* is the first appointment of the pair.

post The state of *local* is changed as follows. The appointment *appt* is deleted from the appointment list (so that there is no appointment for the patient with identity code *pID* at the time slot (d, t)). If the member involved in *appt* is a physiotherapist, then the subsequent appointment is also deleted, so that there is also no appointment for the patient with identity code *pID* in the time slot $(d, t + 1)$. The corresponding diary entry or entries for the member involved in the appointment have the value of the attribute *act* changed to 'F'. If the patient is a member of the practice, then the corresponding diary entries for the patient as member are modified to set *act* to 'F'.

method *getAppt*(*mID* in SeqOfChar, *d* in Int, *t* in Int) **return in Appointment**

pre There is a member with identity code *mID*.

post The returned value is the appointment (as a member) for the day *d* and time slot *t* for the member with identity code *mID*. (If there is no such appointment then a **null** appointment is returned.)

method *getPAppt*(*pID* in SeqOfChar, *d* in Int, *t* in Int) **return in Appointment**

pre There is a patient with identity code *pID*.

post The returned value is the appointment for the day *d* and time slot *t* for the patient with identity code *pID*. (If there is no such appointment then a **null** appointment is returned.)

method *getMAppts*(*mID* in SeqOfChar) **return in mString**

pre There is a member with identity code *mID*.

post The returned value shows all the appointments (as a member) for the member with identity code *mID* in a list.

method *getPAppts*(*pID* in SeqOfChar) **return in mString**

pre There is a patient with identity code *pID*.

post The returned value shows all the appointments for the patient with identity code *pID* in a list.

method *setKept*(*appt* in Appointment, *isKept* in Bool) **state change**

pre *appt* is an appointment in *local*.

If *appt* is with a physiotherapist, then it is the first appointment of the pair.

post The state of *local* is changed by changing the attribute *kept* of *appt* to the value *isKept*.

If the appointment is with a physiotherapist, then the same change is made to the appointment in the following time slot.

method *setNotes*(*appt* in **Appointment**, *n* in *SeqOfChar*) **state change**

pre *appt* is an appointment in *local*.

If *appt* is with a physiotherapist, then it is the first appointment of the pair.

post The state of *local* is changed by changing the attribute *notes* of *appt* to the value *n*.

If the appointment is with a physiotherapist, then the same change is made to the appointment in the following time slot.

method *showDiary*() **return in mString**

pre **true**.

post The returned value shows all the diary entries of *local* in a list.

method *showApptList*() **return in mString**

pre **true**.

post The returned value shows all the appointments of *local* in a list.

method *getAct*(*d* in **Int**, *t* in **Int**, *mID* in *SeqOfChar*) **return in Char**

pre There is a member with identity code *mID*.

post The returned value is the value of the attribute *act* in the diary entry for the day *d* and time slot *t* for the member with identity code *mID*. If there is no such diary entry then the returned value is 'F'.

method *initPractice*() **state change**

pre **true**.

post Some members and patients are added to *local*, as shown below.

Patients

("Vincent", "Vincent-1", **false**)
 ("Harding", "Harding-1", **false**)
 ("Jordan", "Jordan-1", **false**)
 ("Dil2", "Dil-2", **false**)
 ("Dil", "Dil-1", **false**)
 ("Lister", "Lister-1", **false**)
 ("Thomas", "Thomas-1", **false**)
 ("Gabrys", "Gabrys-1", **false**)

Members

("MDil", 'D', "Dil-1", "Dil")
 ("MMeehan", 'D', "Meehan-1", "")
 ("MLister", 'N', "Lister-1", "Lister")
 ("MThomas", 'D', "Thomas-1", "Thomas")
 ("MGabrys", 'P', "Gabrys-1", "Gabrys")
 ("MDil2", 'P', "Dil-2", "Dil2")

Index

class	ISPHYSIO, 6
Appointment, 14	PATFREE, 16
DiaryEntry, 14	PHYSIONOTE, 6
Member, 14	
Patient, 13	relation scheme
	APPOINTMENT, 11
preconditions for <i>addAppt</i> , 15, 29	DIARYENTRY, 11
predicate	MEMBER, 10
ACT, 6	PATIENT, 10
APPOINTMENT, 6	requirements
APPOINTMENTOK, 17	list of, 6
ISDOCTOR, 6	
ISNURSE, 6	transactions
ISPATIENT, 6	list of, 8